MIS 382N: Fundamentals of Neural Networks and Machine Learning

Lecture Notes and Questions of the Day — Fall 2026

Constantine Caramanis
The University of Texas at Austin

Running notes; last updated October 8, 2026

About these notes

These notes follow the lectures, one chapter per class meeting. Each chapter opens with the Questions of the Day that started that class, followed by the discussion of the answers, and then the material covered in the lecture itself. The emphasis is conceptual: the technical details are here when they are needed, but the goal is that you come away understanding *why* we do things, not only *what* the Python command is.

The questions of the day began in Lecture 4. They are a review device: some are quick, some open onto the central conceptual issues of the course. Collected here, they are also a good study checklist. If you can answer every question in this document, with the reason and not just the verdict, you are in good shape for the midterms.

The lecture notes below also reference the Colab notebooks posted on Canvas (under Pages). The notebooks are where you should run things, break things, and change things.

Contents

About these notes **1**

1 Introduction: what this class is about **5**
1.1 Course logistics . . . 5
1.2 What is this AI moment? . . . 6
1.3 Classification and regression . . . 6
1.4 Context, and where we are headed: embeddings . . . 7
1.5 The plan of the class . . . 8
1.6 Supervised learning . . . 8
1.7 Our first models: decision trees, and the idea of a model family . . . 9
1.8 Looking ahead . . . 10

2 Loss functions, model.fit, and looking at the data **11**
2.1 Why supervised learning matters beyond classification . . . 11
2.2 From "search for a good model" to something a computer can do . . . 11
2.3 Loss functions . . . 12
2.4 How this maps to sklearn . . . 13
2.5 Regression, the same way . . . 14
2.6 The Colab notebook: actually looking at the data . . . 14
2.7 Depth 1, depth 3, depth 15 . . . 15
2.8 Looking ahead . . . 15

3 MNIST, training versus testing, and what overfitting is **16**
3.1 Decision trees on images . . . 16
3.2 What model.fit actually does, and why it costs time . . . 16
3.3 Data hygiene: shuffle, check balance, check the labels . . . 17
3.4 Training curves versus testing curves . . . 17
3.5 What the tree looks like . . . 18
3.6 Signal versus noise: a synthetic example . . . 18
3.7 A preview of neural networks . . . 19
3.8 Looking ahead . . . 19

4 Cross-validation, and inventing regression models **20**
4.1 Discussion of the questions . . . 20
4.2 Empirical risk minimization, stated once, carefully . . . 21
4.3 What is missing: choosing the family . . . 22
4.4 Inventing regression models . . . 23
4.5 Where this fits . . . 24

5 Feature engineering, soft scores, and maximum likelihood **25**
5.1 Discussion of the questions . . . 25
5.2 Nonlinear fits with linear regression: the notebook . . . 26

5.3 Soft scores: from regression to probabilities . . . 27
5.4 Where do loss functions come from? Maximum likelihood . . . 28

6 Logistic regression, cross-entropy, and the first look at regularization 30
6.1 Discussion of the questions . . . 30
6.2 Logistic regression: the model . . . 31
6.3 Where cross-entropy comes from . . . 32
6.4 The problem with separable data, and the idea of regularization . . . 34

7 Diagnosing fit, ridge regression, and the fitted model as a random variable 35
7.1 Homework 1 . . . 35
7.2 Discussion of the questions . . . 36
7.3 Regularization, once more . . . 37
7.4 The fitted model is a random variable . . . 38

8 Bias, variance, ridge, lasso, and the road to ensembling 40
8.1 Discussion of the questions . . . 40
8.2 Variance: a rule of thumb worth knowing . . . 42
8.3 Ridge regression in action . . . 42
8.4 Lasso: a different regularizer, a different solution . . . 43
8.5 Toward ensembling . . . 44

9 Bagging, the bootstrap, and random forests 46
9.1 The questions . . . 46
9.2 Where we are . . . 47
9.3 Overfitting is high variance . . . 47
9.4 Averaging reduces variance . . . 47
9.5 Wishful thinking: average the trees . . . 48
9.6 The bootstrap . . . 48
9.7 Why it works . . . 49
9.8 Bagging is not about trees . . . 50
9.9 Random forests . . . 50
9.10 Two questions to close . . . 51

10 Boosting, and the idea of gradient descent 52
10.1 Discussion of the questions . . . 52
10.2 A table of knobs . . . 53
10.3 Boosting: the other end of the depth axis . . . 54
10.4 Boosting for classification: exponential loss . . . 55
10.5 Toward gradient boosting . . . 56
10.6 Interlude: gradient descent . . . 56

11 Gradient boosting 58
11.1 Discussion of the questions . . . 58
11.2 Gradient boosting: boosting for any loss . . . 60
11.3 Bagging and boosting, side by side . . . 62
11.4 On the midterm . . . 62

12 Gradient boosting in practice, hyperparameters, and stacking 63

12.1 Discussion of the questions . . . 63
12.2 Gradient boosting, worked once more . . . 65
12.3 Hyperparameters of boosting, and how to choose them . . . 66
12.4 Ensembling is not about trees . . . 67
12.5 Stacking . . . 67
12.6 Looking ahead . . . 68

13 Stacking in code, the Kaggle setup, and AUC **69**
13.1 Discussion of the questions . . . 69
13.2 How the Kaggle competition works . . . 71
13.3 Scoring imbalanced classification . . . 71
13.4 Looking ahead . . . 73

14 Kaggle strategy, more on AUC, and the first neural network **74**
14.1 Kaggle: a working strategy . . . 74
14.2 Discussion of the questions . . . 76
14.3 Neural networks: the first building block . . . 77
14.4 A question to end on . . . 80

Lecture 1

Introduction: what this class is about

Tuesday, August 25, 2026

Questions of the Day

No questions of the day yet. They begin in Lecture 4.

1.1 Course logistics

This is a standalone class, but I designed it with its continuation in mind: in the spring, the sequence continues with GenAI 1 and GenAI 2. This semester we build all the machinery — classical machine learning, then neural networks, ending with embeddings — that makes generative AI possible to understand.

A bit about me: I teach in Electrical and Computer Engineering, and my research is on the algorithmic and statistical side of machine learning — understanding how large-scale data problems interact with computation. More recently I have been working on language models and diffusion models, which connects directly to where this sequence is headed.

Our TA is Agrim Jain, a second-year master's student in computer science. Office hours (mine and his) will be set once someone sends me your block schedule.

Everything we do will be on Google Colab. Two reasons: it is a *uniform* computing platform, so anything I run works identically for you, with no package-installation headaches; and it gives us free access to NVIDIA GPUs. (Why NVIDIA specifically, when many companies make GPUs? The answer is CUDA — the software bridge between what you type in Python and what actually executes on the hardware. That bridge is what has protected NVIDIA's lead, and it is why your Mac's GPU won't help you train networks in PyTorch.) On Colab's free tier we will comfortably train models with 10 to 100 million parameters, and next semester up to a billion.

Deliverables:

- **Labs.** Hands-on homework in Colab, done in groups. If you make an honest effort, you get full credit — so there is no point in having AI do them for you (it is also against class policy). The labs exist to help you understand the material: run things, break things, change the architecture and see what happens. In class we will only have time to do things one way; the labs are your chance to try the other ways. With neural networks there is very little theory telling us why one choice beats another, so experimenting is not optional — it is how the field actually works.
- **Two in-class midterms**, mid-October and early December. These are conceptual, not computational. Dates are in the syllabus.
- **Kaggle competition.** Coming up fairly soon; it marks our turning point from classical ML into neural networks. It will be binary classification on tabular data, and I predict most of

you will end up using some kind of boosting model. You will have a week. One logistical note: Kaggle limits how many submissions you get per day, and the limit resets daily. Ten hours spread over five days buys you far more feedback than ten hours the night before the deadline. Budget the week accordingly.

- **Final project.** This year I will post a list of potential projects you can choose from; if you choose one of those, the proposal just needs to explain your plan of attack. You are still welcome to propose your own idea, and the scope there is wide: applying models in a new way to an existing dataset is a fine project, but so is compiling, scraping, and cleaning an interesting new dataset, even with only a modest predictive model on top.

Groups. With 22 students, we will have groups of three (plus one group of four or two groups of two). No groups of one, no groups of five. For the final project you may keep your homework group or merge two groups.

1.2 What is this AI moment?

Here is my simplest explanation. Traditionally, how did we go from observations of the world to predictions? We watched what happens around us — planetary trajectories, apples falling from trees — and tried to write down the *laws of nature*. Then we used those laws to predict, say, how a comet moves through a gravitational field. Observation $\rightarrow$ axiomatic knowledge $\rightarrow$ prediction.

AI lives on a different arrow: from observation *directly* to prediction, without extracting axiomatic knowledge in between. This does not mean the 5,000-year-old approach was a bad idea — it just isn't what is happening right now. Instead of watching one apple fall and trying to write the law, we watch apples fall over and over, and predict in a purely data-driven way.

My favorite early example is John Snow and the 1854 cholera outbreak in Soho, London. At the time, the leading theory was that cholera spread through a "miasma" in the air. Snow noticed that the cases clustered around a single water source — the Broad Street pump — a clustering pattern that is statistically inconsistent both with airborne miasma and with person-to-person transmission (the pattern you would see with flu or COVID). He concluded the water was spreading the disease, *before* the cholera bacterium had been identified. No mechanism, no microbiology: a purely statistical, data-driven inference. That is the arrow this class lives on — except that, compared to Snow, we have essentially unlimited data and computation.

1.3 Classification and regression

The two basic prediction problems we will care about are **classification** and **regression**, mostly in computer vision (we will build the framework that extends to natural language, which is where the spring classes pick up).

A running example: you are given a 32×32 color image — which to the computer is just $32 \times 32 \times 3 = 3{,}072$ numbers, the red, green, and blue intensities at each pixel — and you must decide which of 10 categories it belongs to (ship, frog, deer, ...). That is classification; the dataset is called CIFAR-10. We will first attack this problem with classical methods, ensembling in particular, get roughly 40–50% accuracy, and then spend the rest of the semester seeing how close to 100% neural networks can take us.

Contrast that with: given an image of a person, compute the angle at their elbow (say, for pose

estimation). That is regression. What is the real difference? In regression, the possible outputs have a *natural relationship to each other*: a 30° angle is less than 45°, which is less than 60°. In classification there is no such ordering — there is no sense in which "ship" is between "frog" and "deer."

This distinction is worth internalizing, because it is not about whether the output is a number. Recognizing handwritten digits (the MNIST dataset: 28×28 grayscale images of 0 through 9) is classification, even though the labels are numbers — the fact that 9 is bigger than 8 has nothing to do with recognizing a handwritten digit; "9" is just a name. But predicting a person's age from a photo, even restricted to whole numbers, is regression: predicting 50 when the truth is 49 is meaningfully better than predicting 30.

The same problems exist in natural language: deciding whether two paragraphs are about the same topic is (binary) classification; assigning a continuous sentiment score to a review is regression.

1.4 Context, and where we are headed: embeddings

We played a game in class. I showed you a 4×4 grid of colored squares with three squares blacked out, and asked you to predict the hidden colors. You couldn't — every guess was wrong. Then I posed a seemingly *much* harder version: a 1000×1000 image (a million squares, each a continuous mixture of red, green, and blue) with a huge fraction of pixels hidden. And yet a simple algorithm — one that knew nothing special about the picture — reconstructed it well. And you could have, too: with most of a kangaroo visible, you know the hidden region contains the head.

Why is the "harder" problem easier? Context. In the 4×4 grid, the 13 visible squares are context too, but we don't know the rules that generated them. In the photograph, we bring an enormous amount of knowledge from the millions of images we have seen in our lives: we recognize objects and their parts (tail, legs, so the head is what's missing); we know the color palette; and even if you had never seen a kangaroo — or any animal with a tail — you would still know that objects have edges, and that pixel values change sharply across an edge. Note these are "rules" in quotes: nobody hands us a written list, and that is exactly the magic. If we had an explicit list of rules to check, we would be back on the axiomatic arrow, back to Euclid. The same story holds for language: fill in the blacked-out word in a sentence. In principle you are choosing among some 50,000 English words, yet it is easy — because of context.

Here is the destination this class drives toward, and the single most important idea of the semester: **semantic embeddings**. Two images of kangaroos are, pixel by pixel, completely different — if you subtract raw pixel intensities, one kangaroo is probably closer to a koala photo than to the other kangaroo. Yet to us they are obviously similar. So we will build a transformation — a box — that maps an image (or a word) to a vector in a different geometric space, where *geometric* proximity coincides with *semantic* proximity. Two kangaroos map to nearby points; a kangaroo and a koala map to distant points; "kangaroo" and "marsupial" map close together as words. For images we will use something like 512-dimensional vectors. Once we have that, hard questions become easy: "are these two images similar?" becomes "what is the distance between two points?" — which is trivial for a computer.

Embeddings are the endpoint of this class and the bridge to generative AI. Describing what belongs in the hidden region of the kangaroo image is itself a classification problem (it's not a car wheel, it's not a lollipop — it's a kangaroo head); and being able to *describe* what goes there is the same thing as being able to *generate* it. These ideas power style transfer (render this content in the

style of that reference image — no one has to articulate in rules what "the style" is), denoising and deblurring, and important applications: my colleague Jon Tamir uses exactly these ideas in magnetic resonance imaging, where ideal images require patients to stay perfectly still — impossible for children, or for organs that move, like the heart — so one must reconstruct sharp images from blurred measurements.

One project we will do together in this class: classifying breast-cancer ultrasound images as malignant, benign, or normal, from a few thousand examples. We will build larger and larger neural networks, watch accuracy climb — until it stops climbing. Then comes a main theme of the course: once you have built the best network you can for your data, where else can you find classification power? (The answer, eventually: transfer learning and embeddings.)

1.5 The plan of the class

Roughly: supervised learning and empirical risk minimization; classification and regression with classical models, including logistic regression; ensemble methods, which are the workhorses of classical ML; and — a drum I will beat all semester — how to tell whether your model is any good. Then neural networks, transfer learning, and embeddings, which hand off to the spring: transformers, language models, multimodal models, RAG.

On evaluation, one point deserves emphasis now. Python will *always* give you an answer. You will always get a model, and it will always produce predictions. It is entirely our job to understand how good that answer is — how the model will perform when it is released and running live, on data it has never seen. Sometimes the honest answer to your company will be "our data cannot support what we need." And sometimes 70% accuracy is superb: a doctor telling me I am twice as likely as average to develop heart disease is actionable information, even though it is far from a certain prediction.

1.6 Supervised learning

The whole name of the game is **learning from examples**. Rather than being told the principles that make a cat a cat, we see many labeled examples: here is a data point, here is its label. Formally, we observe n labeled pairs (x_i, y_i), and we want to learn the relationship — a function h that takes an x (a bunch of numbers) and outputs a prediction $\hat{y}$. The hat matters: it marks quantities computed from data. The x's are often called *features*, especially in classical ML where they are interpretable quantities like age or blood pressure — unlike pixel intensities. And the goal, always, is to predict well on data the model *has not seen*, not on the data it was trained on.

Our first dataset is a diabetes-diagnosis problem: roughly 800 patients (one per row — and remember Python indexes from zero), 8 features per patient that are relatively easy to measure, and an outcome column, diabetic or not, that is harder to measure — that is precisely why predicting it is valuable.

Before building any model: *look at the data*. The temptation to feed the dataset straight into a model and read off the output is enormous; resist it. We can't plot 8-dimensional data, but we can plot two columns at a time. Plotting glucose against BMI, colored by outcome, one thing jumps out: the two classes **overlap heavily**. That single observation is worth a lot. It tells us that from these two features alone, no honest algorithm can be 95% accurate — so when we later build one that *is* 95% accurate on glucose and BMI, we will know exactly what it is doing: overfitting. Overlap

doesn't mean the data are bad; it means we should calibrate expectations, the way a doctor can tell you your risk is elevated without pretending to certainty.

Contrast this with the image problem. There, 50% accuracy across 10 classes is five times better than random guessing — and still not good enough. Why? Because we already know a classifier that is nearly 100% accurate on those images: *us*. If a human can tell the boat from the deer, the signal is in the data, and a model that confuses them simply isn't good enough yet. This is the kind of reasoning — what should we expect, when do we stop, what do we do next — that the technical machinery alone will never give you.

1.7 Our first models: decision trees, and the idea of a model family

Think of a classifier on two features (x_1, x_2) as a *partition of the plane*: on one side of a boundary, predict $\hat{y} = 1$; on the other, $\hat{y} = 0$.

A **decision tree** is a model that may only cut the space with vertical and horizontal lines. A depth-1 tree makes one axis-aligned cut, and specifying it requires exactly four decisions: which coordinate to cut on (vertical or horizontal), where to put the cut, and what to predict on each side. Four parameters. A depth-2 tree splits each resulting region again: ten parameters. And so on — a depth-d tree carves the plane into up to 2^d regions.

That exponent should make you slightly nervous. A depth-10 tree gives $2^{10} = 1{,}024$ regions — and our diabetes dataset has only about 700 points. So as long as no blue and red point sit exactly on top of each other, a depth-10 tree can get essentially 100% accuracy *on the training data*. That is not a triumph; it is memorization, and it says nothing about accuracy on data the tree hasn't seen.

I will say it bluntly: on their own, decision trees are very weak models — they either fail to capture enough structure or they overfit, with no sweet spot in between. They have exactly two redeeming qualities, and both matter enormously. First, they are *interpretable* — they match how humans reason. "Your glucose is over 140, so you are at elevated risk" is an explanation you can act on, unlike "a million-parameter network assigns you 80%." Second, they can be *combined* into powerful models — that is ensembling, the centerpiece of the classical portion of this course.

Now compare with **logistic regression**, which (we will define it properly soon) draws a single straight line at any angle:

$$\hat{y} = \text{sign}(\beta_0 + \beta_1 x_1 + \beta_2 x_2),$$

three parameters, and varying them produces every possible linear cut of the plane. Which family is richer? Logistic regression is strictly richer than depth-1 trees: anything one axis-aligned cut can do, a line can do. But depth-2 trees and logistic regression are *incomparable*: no tree of any depth can produce a diagonal cut, and no single line can reproduce a depth-2 tree's four-quadrant partition. Some models simply cannot express some functions.

Here is the point of this whole discussion, and it is the conceptual template for everything to come. "A decision tree" or "logistic regression" is not a model — it is an *infinite family* of models, a declaration of which functions h you are allowed to consider. Learning then means algorithmically picking the best (or at least a good) member of that family, by choosing the parameters. It has to work this way: without restricting to a parametric family, there is nothing for an algorithm to search over. And this is exactly what a neural network will be, too — choosing an architecture is like choosing "depth-2 decision trees" (it fixes the family and the number of parameters), and training is like picking which particular tree (it fixes the weights).

1.8 Looking ahead

Next class we pick this up in Python: Colab, the diabetes dataset, our first models, and the question of what "the best model in the family" actually means.

Lecture 2

Loss functions, model.fit, and looking at the data

Thursday, August 27, 2026

Questions of the Day

No questions of the day yet. They begin in Lecture 4.

2.1 Why supervised learning matters beyond classification

Last time we introduced supervised learning and, informally, empirical risk minimization: choose a family of models, then find a good member of that family. For now these are tools for solving classification and regression problems. But they are more fundamental than that. Even in generative AI, where the task does not look like classification or regression and there is no obvious loss function, the main tool for building models is supervised learning. The reason is practical: anything done at scale has to run on GPUs, and the technical link to GPUs is an algorithm called *gradient descent*. To use gradient descent, a problem must be framed as "search over parameters to minimize a loss." That framing is exactly what we are building now, and it will return again and again.

A note on notation. We see pairs (x_i, y_i). I will call x the *point* or the *features* and y the *label*; statistics calls them independent and dependent variables, and every field that has used this kind of modeling has its own names. The model h is a function in the high-school sense: numbers in, numbers out, deterministic computation. That is true even when the input is an image and the output is "cat." The image is 3,072 pixel intensities, and h multiplies, adds, and thresholds them. When we build neural networks you will see this concretely.

2.2 From "search for a good model" to something a computer can do

Looking at the glucose-versus-BMI scatter plot and saying "there seem to be more blue points down there" is not an algorithmic thought. To search for a good model with a computer, we need two things.

First, we need to describe *all* the candidate models, in a form a program can enumerate. Writing $h(x_1, x_2)$ is useless: it encodes infinitely many objects of unbounded complexity. Writing "depth-1 decision trees" is useful: even though there are infinitely many of them, every one is described by four numbers (which coordinate to split on, where to split, and what to predict on each side). A program can search over four numbers, even by brute force. This is what we mean by a **parametric family**.

Second, we need to say what "good" means, in a form a program can evaluate. If you propose one model and I propose another, how do we decide who is right? The obvious answer is to count mistakes. That is a fine answer, but it is not the only one, and the alternatives matter.

A question came up here that is worth recording: what does it mean for glucose and BMI to have "predictive power"? Knowing these two features, we can do better than random guessing, so they carry signal. They do not carry enough signal to predict perfectly, and nothing in the data tells us whether some *other* feature would. The right way to frame every such question is in terms of the decisions available to you. If you cannot collect more data, do not ask "what feature do I wish I had?"; ask "what is the best I can do with this, and how reliable is it?" If you can talk to a clinician and go back to the hospital, the question changes.

2.3 Loss functions

The way we make "good" precise is a **loss function**. A model is better than another if it has lower loss, and every loss we use will have the same shape: a sum over the data points of a per-point penalty that compares the true label with what the model said,

$$\text{Loss}(h) \;=\; \sum_{i=1}^{n} \ell\big(y_i, h(x_i)\big).$$

This form is flexible. The per-point penalty ℓ can charge differently for different kinds of mistakes, which is exactly what you want when a false negative on a rare, aggressive cancer costs far more than a false positive that costs someone a few nights of sleep and a follow-up exam.

We looked at four choices for classification.

0/1 loss. $\ell = 0$ if the prediction matches the label and 1 otherwise. Summing it counts mistakes; dividing by n gives error rate, and one minus that is accuracy. This is the most natural loss and, as a student pointed out immediately, it can be badly misleading. If a disease affects a fraction of one percent of the population, the model that says "you don't have it" without looking at the data is extremely accurate and completely useless.

Hinge loss. Not every mistake is equal. A point misclassified just barely across the boundary was close to a coin toss; a point misclassified far from the boundary is a confident, bad error. Hinge loss charges nothing for correct points and charges misclassified points in proportion to their distance from the boundary.

Exponential loss. Suppose the model does not just say red or blue but produces a score z whose sign gives the prediction and whose magnitude reflects confidence. Exponential loss charges e^{-z} for a correctly classified point (so a correct but timid prediction still costs something, and only a confident correct prediction costs nearly zero) and charges $e^{+|z|}$, growing very fast, for a confidently wrong one. This loss favors models that are *confidently correct*, and punishes being confidently incorrect severely. A model trained under it would rather make many small mistakes than one large one.

Cross-entropy loss. Qualitatively like exponential loss: zero loss only for confident, correct predictions; a large penalty for confident, incorrect ones; a gentler ramp than the exponential. We have not written its formula yet. This is the loss we will use for almost all our neural network work, and in Lecture 6 we will see where it comes from.

The picture to keep in mind is the *loss curve*: plot the loss assigned to a single point as it moves from confidently correct, to barely correct, to barely wrong, to confidently wrong. 0/1 loss is a step: zero, then one. Hinge is zero then a ramp. Exponential and cross-entropy start small but positive, then rise steeply. These curves are the qualitative fingerprint of each loss.

Which loss is best?

None of them. If one were best I would not be showing you four. There is no theorem that cross-entropy beats exponential loss. What there is, is a modeling task: you know what qualities you want in a solution — good for the patients, good for the customers, good for the business — and you have to translate that into something a computer can evaluate. Different losses encode different preferences. On the synthetic example in the slides, sweeping over all vertical partitions, the four losses picked four different boundaries. **Changing the loss changes the notion of "better," and therefore changes the model you get.** If your application has a particular character you want to bring out, your job is to find the loss that encodes it.

A remark on computation

Plotting loss against the position of the boundary for each of the four losses shows one more difference that is qualitative rather than a matter of taste. The 0/1 loss curve is jagged (it jumps every time the boundary crosses a data point), and it goes *up* in places on the way to the minimum. The other three are smooth and, in this example, have a single valley.

Why care? Because at scale, the only search we can afford is a *local* one: I am here, should I move a little left or a little right? That is what gradient descent does. On a smooth, single-valley loss, local search finds the minimum. On the 0/1 loss, local search gets stuck on one of the little upticks and declares victory. This is why 0/1 loss is notoriously hard to optimize, and why we will always train on a smooth surrogate (cross-entropy, typically) even when what we ultimately report is accuracy. With four parameters we can brute-force; with ten million we cannot.

2.4 How this maps to sklearn

The two-step logic — choose a family, then choose the loss and find the best member — is mirrored in the syntax of `sklearn`:

`model = DecisionTreeClassifier(max_depth=1)`	choose the family (and, implicitly, the loss)
`model.fit(X, y)`	search the family for the member with lowest loss on (X, y)
`model.predict(X_new)`	evaluate the chosen member on new points

Notice what the first line does *not* do. It does not see any data. It cannot choose a good model; it only declares which models are allowed. The second line is where the data enter and where the four parameters of the depth-1 tree get chosen. After it runs, `model` is no longer a family but a specific function, and `predict` evaluates it: for a new point $(88, 40)$, the tree checks whether the first coordinate exceeds 147.5, it doesn't, so it goes left and says blue.

Where did the loss go? It is packaged into the family. Whoever wrote the decision tree code chose a default loss and exposed a few alternatives as options (we will see one, `criterion`, next time). If

you want a loss they did not anticipate, you rewrite the package, including `fit`.

One more thing this code hides: `X` is whatever you pass. If you pass the whole 8-column matrix, the tree splits on any of the eight features. If you first select glucose and BMI, it uses those two. Should skin thickness = 0 have stayed in? The code will run either way and will always return an answer. Every decision it hides is one you have to make.

2.5 Regression, the same way

The paradigm is identical for regression. Take a scatter of points and try to learn y from x. "Fit a curve" is not algorithmic; "fit a line through the origin" is: the family is $h(x) = \beta x$, one parameter. For the loss, a common choice is **squared loss**, $\ell(y, \hat{y}) = (y - \hat{y})^2$, the squared vertical distance from each point to the line. Then

```
model = LinearRegression(fit_intercept=False),    model.fit(X, y)
```

means: find β minimizing $\sum_i (y_i - \beta x_i)^2$. For this one-parameter quadratic you could do it by hand: take the derivative in β, set it to zero, solve. We won't be doing such calculations in this class, but it is worth seeing once what `fit` is doing behind the curtain. Add an intercept and the family becomes $\beta_0 + \beta_1 x$, two parameters, same story.

Whether we are fitting a line or partitioning a plane, the recipe is: state the allowable models, state what makes one better than another, and let `fit` search.

2.6 The Colab notebook: actually looking at the data

The rest of class was in the first notebook (posted on Canvas under Pages; save a copy to your own Drive, since the shared version is read-only). A few things worth recording.

Libraries. `pandas` for loading and manipulating tabular data (CSV files especially), `numpy` for numerical computation, `matplotlib` for plotting, `sklearn` for classical ML. I paste the same import block into every notebook. One recurring pain point: these libraries also define *data structures*, and not every algorithm accepts every structure. `sklearn` and `matplotlib` want NumPy arrays, not pandas DataFrames; later, PyTorch will want its own tensors. Converting between them (`.values` turns a DataFrame into an array) is conceptually trivial and a very common source of errors. Also: fix the random seed so that randomized algorithms give the same result every run.

Colab forgets. Every time the runtime resets, all files are gone; you re-download the data each session. Commands like `gdown` make this manageable.

Look before you model. Printing the DataFrame told us there are 768 patients and 9 columns. Counting the labels told us about 500 are non-diabetic and about 268 are diabetic. That number sets a *baseline*: guessing "not diabetic" for everyone is right about 65% of the time without looking at any features. Any model we build has to beat 65% to have done anything. For CIFAR-10, which is balanced with 10 classes, the baseline is 10%; for ImageNet with 1,000 classes it is 0.1%, so a 10%-accurate ImageNet model is a hundred times better than guessing. Always know your baseline.

The scatter plot had a surprise. About 20 points sit at glucose = 0 or BMI = 0. Nobody has zero BMI. These could be data-entry errors, scanner errors, or blanks that were converted to zero because a CSV cell has to contain something. We didn't collect the data, so we don't know. We would never have noticed this if we had converted to NumPy and run `fit` immediately.

What to do about it? Three options came up in discussion:

1. *Filter*: drop those patients. Simple, and reasonable here (768 becomes about 750).
2. *Impute with a summary*: replace the missing BMI with the mean, median, or mode of the column.
3. *Impute by prediction*: BMI is probably correlated with the other features, so predict the missing value from them.

Which is better? As always in this class, that is a trick question until you say what "better" means for your application. Filtering is attractive in this toy problem, but think about a real medical record: many columns, and very few of them measured for *every* patient. Drop every patient with any missing value and you have no patients left. The zero-BMI patient still has seven other measurements that carry information, and imputation is how you keep them.

The same reasoning applies to the one point with BMI far above everyone else. Should it stay? If you want a model that is accurate for people with BMI between 10 and 50, remove it. If you want a model that is accurate out there too, keep it. Python won't tell you; the textbook won't tell you. You have to know what product you are building and for whom. Data cleaning is unglamorous and it is usually the largest part of the job.

2.7 Depth 1, depth 3, depth 15

After filtering, we trained a depth-1 tree (the "decision stump" at glucose = 127.5), then depth 3 (77% training accuracy), then depth 15 (97%). Then we plotted the depth-15 decision regions.

They were absurd. A patient with very high glucose is predicted non-diabetic, but if she lowered her glucose a bit the model would call her diabetic. Another patient is diabetic unless he *raises* his BMI slightly. This is overfitting, and here we can see it with our eyes because there are two features and we understand what glucose and BMI mean.

The question that closed the class: how do we detect this *algorithmically*, when the data are 700-dimensional or the features mean nothing to us? Training accuracy is useless for this purpose; it will keep rising with depth. What we want is more data — data the model has never seen — and if we can't collect more, we pretend. Hold out a quarter or a third of the data, fit on the rest, and measure accuracy on the held-out part. This **train/test split** is the first tool for the question that this whole course keeps returning to: how good is my model on the data I have not seen?

2.8 Looking ahead

Next time: MNIST, training versus testing curves, and what "overfitting" looks like when you cannot plot the decision regions.

Lecture 3

MNIST, training versus testing, and what overfitting is

Tuesday, September 1, 2026

Questions of the Day

No questions of the day yet. They begin in Lecture 4.

3.1 Decision trees on images

Today's dataset is MNIST: 70,000 grayscale images of handwritten digits, each $28 \times 28 = 784$ pixels, each pixel an intensity between 0 and 255 (we rescale to $[0, 1]$). To `sklearn`, an image is a row of 784 numbers, exactly as a patient was a row of 8 numbers. The label is the digit, 0 through 9, and as we discussed on the first day this is a 10-class classification problem, not regression.

What does a decision tree do with such a row? The same thing it did with glucose: it picks one coordinate — now a single pixel — and asks whether its intensity is above or below a threshold. A depth-1 tree looks at one pixel and can output at most two different digits, so it cannot possibly exceed about 20% accuracy. A depth-2 tree looks at up to three pixels and outputs at most four digits. In general, depth d gives at most 2^d leaves, hence at most 2^d distinct outputs.

Splitting on individual pixels does not sound like a good idea, and in general it isn't. It works tolerably on MNIST (trees reach about 70% test accuracy) for one reason: the digits are *centered*. A bright pixel dead center means you are not a 0; a bright pixel in the far corner means you are not a 1 or a 7. Because every image is registered the same way, a fixed pixel carries consistent information across images. If the digits were shifted around at random, or if we added random noise, this logic would collapse. So the right way to say it is not "centering is a good first step" (though it is) but "MNIST is easy *because* it is centered." On real problems — scanning an envelope at the post office — finding and centering the digit can be as hard as reading it.

3.2 What model.fit actually does, and why it costs time

It is worth walking through what `fit` does for a depth-1 tree on MNIST, because it explains where computation goes. The tree must choose a pixel, a threshold, and a label for each side. The algorithm picks the upper-left pixel, tries threshold 0.01, sweeps through all 60,000 training images to compute the accuracy of that split (for each way of assigning labels to the two sides), then increments the threshold and sweeps again, and again for every threshold, and then moves to the next pixel and repeats all of it. Hundreds of thresholds, 784 pixels, a full pass over the data for each. The output is the single best pixel-threshold-labels combination.

Two lessons. First, the cost of `fit` scales with the amount of data: ten times the data, ten times the time per pass. Second, nobody works this hard to solve MNIST, but the logic — "I need to run many experiments, each of which passes over all the data" — is exactly what will bite us with neural networks. The difference between three minutes and thirty minutes per run is the difference between trying twenty things in an afternoon and trying two. So we will constantly be looking for compute bottlenecks and for ways to get a rough answer cheaply.

The simplest such trick: subsample. Train on 600 images instead of 60,000 and every run is a hundred times faster. The cost is lower accuracy, and that is fine while you are debugging and exploring. The *final* run uses all the data and can take as long as it needs; what you cannot afford is a ten-hour turnaround every time you want to test an idea.

3.3 Data hygiene: shuffle, check balance, check the labels

Three small things from the notebook that are not small in practice.

Shuffle the data. If the downloaded file happens to be sorted by label and you take the first 80% for training, you train on 0–7 and test on 8–9, and get 0% test accuracy. Always shuffle before splitting.

Check the class balance. MNIST is roughly balanced, so the random-guess baseline is about 10%. Know this number before you are impressed by anything.

Check how the labels are stored. My first attempt at counting labels reported zero nines, which is obviously wrong for 70,000 digits. The labels were stored as strings, not integers, and my comparison silently failed. These are boring problems, and they are dangerous precisely when the error is *not* obvious. Here I knew there had to be nines; on a dataset you don't understand you might not.

3.4 Training curves versus testing curves

The main experiment: split MNIST into training and test sets, train decision trees of depth 1 through 15, and for each depth record the accuracy on the training data and on the test data. Plot both against depth.

The shape is typical and you should be able to sketch it from memory. The two curves start together at low accuracy. Training accuracy climbs steadily toward 100%. Test accuracy climbs with it for a while, then the two separate: training keeps rising, test flattens out around 70%. The **gap** between them is the signature of overfitting.

Why is training on top? Because the depth-13 tree was fit *to* the training data. Anything it chose to memorize about those particular images helps it there and helps not at all on new images.

This is the moment to draw a distinction that we will return to all semester: what is *always* true versus what is *typically* true.

- *Training accuracy is non-decreasing in depth.* Always. A depth-k tree is a special case of a depth-$(k+1)$ tree — the one whose extra splits are degenerate (everything goes to one side). `fit` searches over all depth-$(k+1)$ trees to minimize training loss, so it can never do worse than the best depth-k tree. Give the optimizer strictly more power and the same objective, and it will not choose to hurt itself.

- *Test accuracy is above training accuracy.* Typically not, but possible. The split is random. It is conceivable, though unlikely, that the test set happened to contain the easy examples. Whenever there is a random step in the pipeline, ask whether the pattern you see is guaranteed or merely likely.
- *Test accuracy increases with depth.* Sometimes, and it can also decrease. The extra layer was added to improve training loss; nothing about that construction says anything about a different dataset. On MNIST the test curve flattened; on the synthetic example below it came back down.

A good question from class: on the test curve, do I want the depth where test accuracy peaks, or the earlier depth where the two curves first separate? If the test set is large (we tested on 10,000 to 20,000 images) the test curve is a trustworthy estimate of performance on new data, and if accuracy on new data is the only thing you care about, take the peak. But be aware that a model out there is carrying artifacts of memorizing the training set. If that matters to you — for interpretability, for robustness to shifts in the data — prefer the earlier point.

3.5 What the tree looks like

We printed a depth-5 tree trained on 20 images. The root asks whether pixel 544 (somewhere in the lower third of the image) is below 0.275; 11 images go left and 9 go right; each side splits again, and so on to depth 5. It is completely readable. The depth-15 tree trained on the full data took a long time to render because it has on the order of 30,000 leaves. With 30,000 leaves and 60,000 training images, the tree can afford to devote a leaf to a handful of images that have a stray mark in some corner. That is memorization, and it is exactly what makes the training curve climb while the test curve stalls.

3.6 Signal versus noise: a synthetic example

Real data are more interesting, but to isolate a phenomenon it helps to build data where you know the truth. I made a two-dimensional dataset: points with $x_1 < 5$ are labeled red and points with $x_1 > 5$ are labeled blue, and then I flipped a random 10% of the labels.

There are now two kinds of pattern in the training data. The left-right split is a *trend*: it is present in the training set and it will be present in any test set drawn the same way. Learning it helps you tomorrow. The specific flipped points are *noise*: which points got flipped in training has no bearing on which get flipped in the test set. Learning that "this particular blue point in the red region is blue" helps you not at all tomorrow.

The shallow tree found the vertical line near $x_1 = 5$. It did not find the flipped points, not because we told it they were noise (we didn't, and in a real problem we couldn't) but because it only had one cut to spend and that cut was worth the most. The deep tree found the same line *and* carved out little boxes around the flipped points. Its training accuracy is higher. Its test accuracy is lower. This is the case where the test curve turns down.

Two remarks. First, here we cannot appeal to common sense the way we did with glucose and BMI: x_1 and x_2 mean nothing, and the deep tree's carved-up regions do not look obviously wrong. The *only* thing telling us the shallow tree is better is the train/test split. Second, a student asked how test accuracy could exceed 90% when 10% of labels were flipped. Because a test set is a finite

sample, and in a finite sample anything can happen; 10% is the rate at which nature flips coins, not a guarantee about any particular draw. This is the same "always versus typically" distinction again.

Data drift

I have been using the word *trend* rather than only *overfitting* on purpose. Memorization is the simplest way to fit something in today's data that is absent tomorrow, but it is not the only way. A model predicting which customers return to your store stops working one day; something about the population changed, and a pattern that held in your historical customers does not hold in the new ones. That is *data drift* (or distribution drift). The whole train/test framework rests on an assumption: the data you test on look, statistically, like the data you will see tomorrow. When that assumption fails, no amount of cross-validation will save you. Keep this in the back of your mind every time a model is deployed.

3.7 A preview of neural networks

I ended by fitting a small two-layer neural network to the same synthetic data (we will learn every line of that code later). The network has an enormous capacity to find trends, and an equally enormous capacity to memorize. Its decision regions wrapped around individual flipped points just as the deep tree did. How would you know that was memorization if you couldn't plot it? Same answer as always: the train/test split. Something is useful only if it is useful on data the model has not seen. That one sentence is the whole story of this part of the course.

Why did the network not always find the flipped points, when it clearly had the capacity to? Two possibilities, and we will return to both: either the architecture lacks the flexibility, or — more interesting — the training procedure, in the time we gave it, simply did not find that solution. With neural networks, "the best model in the family" and "the model that training returns" are not the same thing.

3.8 Looking ahead

Next time: cross-validation, a better version of train/test. Then ensembling, then boosting, and then on to neural networks.

Lecture 4

Cross-validation, and inventing regression models

Thursday, September 3, 2026

Questions of the Day

Q1. True or False: Predicting a handwritten digit from 0 through 9 is a regression problem because the possible labels are numbers.

Q2. True or False: On a highly imbalanced data set, a classifier can achieve high accuracy while still being useless for the application.

Q3. True or False: Once the model family and training data are fixed, changing the loss function cannot change which model is selected as the best model.

Q4. Free answer: A researcher decides that data points with the largest values of feature 3 are hurting the model's fit to the training data, so they remove them. After deployment, the model performs worse. Explain how removing those data points could cause this outcome.

Q5. True or False: Increasing the maximum depth of a decision tree cannot reduce its training accuracy, but it can reduce its testing accuracy.

Q6. True or False: Testing accuracy can never be higher than training accuracy.

4.1 Discussion of the questions

Starting today, each class opens with review questions. Some are quick; some lead into the important issues. Homework teams take turns answering.

Q1. False. The labels 0 through 9 are names, not quantities. Nothing about recognizing a handwritten "9" depends on 9 being larger than 8; the problem is no different from telling a cat from a boat from a deer in CIFAR-10. Contrast: guessing a child's age from a photo, rounded to a whole number 0 through 9, *is* regression, because there 5 really is more than 4 and guessing 5 for a 4-year-old is better than guessing 9. The test is whether the labels have a numerical relationship to each other, not whether they are written with digits.

Q2. True. The first answer offered was "the test data might not be representative," which is a real problem but a different one; let us insist that training and test data have identical statistics. Even then: if a disease is present in 0.1% of patients, the classifier that tells every patient "you don't have it" is 99.9% accurate on training, on test, and on every future patient, and is worthless. The key word is *imbalanced*, which refers to the distribution of labels. Rare-disease screening, rare-event detection in particle physics, fraud — these are all imbalanced problems where accuracy is the wrong metric. This is why checking the label balance was one of the first things we did with the diabetes data: two-thirds of those patients are non-diabetic, so 66% accuracy is the do-nothing

baseline. If someone tells you they shuffled, split, and got 99% accuracy, do not be impressed until they tell you the base rate.

Q3. False. Changing the loss changes what "best" means, and so it changes which model `fit` returns. We saw this on the synthetic example in Lecture 2: counting mistakes, hinge, exponential, and cross-entropy each picked a different boundary. The follow-up question is which loss to choose, and the answer is that there is no universal one. It depends on what mistakes cost in your application. For one diagnosis a false negative is catastrophic; for another, the false positive is what hurts. A doctor friend explained why we don't screen everyone for lung cancer with CT scans even though it would catch some cancers: the false positives lead to needle biopsies, and biopsies sometimes collapse a lung. The costs of the two kinds of errors are what an expert brings to the table, and the loss function is how you express those costs to the algorithm.

Q4. The student's answer was essentially right and only the jargon was off: removing the high-feature-3 points changed the distribution of the *training* data, so the model was trained on a population different from the one it was deployed on. ("Imbalance" usually refers to labels; the right words here are *distribution shift* or *data drift*.) Everything we do assumes the data we train on look like the data we will see tomorrow. Break that assumption and the test-set estimates are no longer about the right population.

Two further points came out of the discussion. First, the finding itself is valid and potentially valuable: the model really does fit better on the narrower population. Second, the value can only be captured by deploying on that narrower population. A house-price model for Austin that ignores the handful of $7 million houses can be more accurate for the rest of the market; that is a legitimate design choice, so long as you do not then use it to price a $9 million house. Building a very good model for a slice of the population is fine. Applying it outside the slice is not.

Q5. True. Both halves. Training accuracy cannot drop because a depth-$(k+1)$ tree can always reproduce the depth-k tree by making its extra splits degenerate; `fit`'s mandate is to find the best model in the family on the training data, and with more options it will not settle on a worse one. Test accuracy can drop because the extra splits were chosen to fit the training data, and what they fit may be noise. The word for that is overfitting.

Q6. False, and this is a trick question. We *expect* training accuracy to be higher, since the model was optimized for the training set and not the test set. But "can never" is absolute, and the split is random. It is possible, by luck, for the test set to be easier. Recall the synthetic example with 5–10% of labels flipped: if by chance none of the flipped points land in the test set, test accuracy can be 100% while training accuracy is not. Unusual, but possible.

4.2 Empirical risk minimization, stated once, carefully

The process we have been describing has a name: **empirical risk minimization** (ERM). Jargon is worth knowing only so you recognize it when others use it, but it is worth writing the process down once with symbols, because the same four steps will describe training a 10-million-parameter neural network.

1. **Get data** $(x_1, y_1), \ldots, (x_n, y_n)$.
2. **Choose a parametric family** $\{h_\theta\}$. Here θ stands for whatever numbers specify a member of the family: for a depth-1 tree, the four numbers (which feature, what threshold, what to predict on each side); for a depth-2 tree, ten numbers; for logistic regression on two features,

three; for ResNet-18, about ten million. We use one letter because we never want to enumerate them, only to say "search over all values of θ."

3. **Choose a loss** $\ell(y, \hat{y})$: cross-entropy for classification, squared loss for regression, or whatever encodes what you care about.

4. **Fit:**

$$\hat{\theta} \;=\; \arg\min_{\theta} \sum_{i=1}^{n} \ell(y_i,\, h_\theta(x_i)).$$

"$\arg\min_\theta$" is shorthand for: search over every allowable θ, and for each one compute the total loss by going through the data point by point and adding up the per-point losses; return the θ with the smallest total.

In `sklearn`, steps 2 and 3 are packaged together in `model = LogisticRegression()`, or `model = DecisionTreeClassifier(max_depth=3)`, and so on; step 4 is `model.fit(X, y)`. I separate them here because for neural networks we will have to specify the model and the loss as separate steps, and write the fitting loop ourselves.

The key structural fact, which we will exploit later: the loss is a *sum over data points*. Accuracy is a sum (of 0/1 losses) divided by n. Every loss we use has this additive form.

4.3 What is missing: choosing the family

ERM tells you how to find the best depth-3 tree. It does not tell you whether to use a depth-3 tree, a depth-4 tree, or logistic regression. And you cannot answer that by running `fit` on each and comparing training loss: the more flexible family always wins on training data, as Q5 just reminded us. What we want is the family that, *when trained on the training data, does best on data it has not seen*. Our answer so far is the train/test split: fit each candidate on the training portion, evaluate each on the test portion, pick the winner.

Cross-validation is a more data-efficient version of exactly this idea. Instead of one split, cut the data into K pieces (say $K = 6$; nothing is special about 6). Then, for each k from 1 to K: hide piece k, fit the model on the other $K - 1$ pieces, and record the loss C_k on the hidden piece using `model.predict` on the hidden X_k. Do this K times, each time hiding a different piece. The **K-fold CV score** is the average $\frac{1}{K}\sum_k C_k$.

A single train/test split is what you get if you compute only C_6 and stop. Cross-validation uses every piece as the test set once, so every data point contributes both to training and to evaluation, and the random luck of one particular split averages out. It gives a more accurate estimate from the same data. The price is computation: K fits instead of one.

What is it *for*? For exactly the question ERM does not answer. Should I use depth 3 or depth 4? Compute the CV score for depth-3 trees, compute it for depth-4 trees, take the better. Depth 3 or logistic regression? Same procedure. Boosting with 200, 250, 300, or 350 estimators (a hyperparameter we will meet later, which behaves like depth in that training accuracy always rises with it)? Same procedure. Whenever the question is "which family?" or "which setting of a knob that controls model complexity?", cross-validation is the answer. If you have already committed to a single family and a single setting, there is nothing to cross-validate.

A student asked whether this resembles bagging. It doesn't: bagging resamples the data *with replacement* to create many datasets of the same size, in order to build an ensemble. Cross-validation

partitions the data to estimate how a model will do on new points. We will get to bagging.

Bigger K or smaller K?

Bigger K means each fit sees more data ($\frac{K-1}{K}$ of it) and the average is over more folds, so the estimate is statistically better. It also costs K times the computation of a single fit. For decision trees on the diabetes data, 100 times a negligible cost is still negligible. For neural networks, where one fit may take ten minutes or ten hours, we will not use cross-validation at all; it is too expensive. Statistically better, computationally worse.

Look at the spread, not only the average

A good suggestion from class: look at the individual fold scores $C_1, \ldots, C_K$, not just their mean. If two models have the same average but one has scores that vary wildly across folds, be worried about that model. Each fold was a simulation of "train on data like this, test on new data like this," and if the outcome swung from fold to fold, you have low confidence in what will happen on tomorrow's batch. (This became Q4 of the next class.)

4.4 Inventing regression models

I want to reinvent the wheel for a while: forget the tools we have and ask what we actually want from a model.

Example: heart disease from BMI and blood pressure. A shallow decision tree gives "high risk" or "low risk." That is fine, but it leaves something on the table. My doctor does not say "you will definitely develop heart disease"; she says something with nuance. A model that bins customers into "will buy" and "won't buy" does not tell me how much effort to spend on each customer. We would like a probability. Trees do not naturally give one. Hold this thought; it leads to logistic regression next week.

Example: house prices. You can scrape the tax assessor's site for square footage, bedrooms, and sale price; Zillow (I believe) added a *walkability* score. The goal is to predict sale price. A first model, **linear regression**:

$$\widehat{\text{price}} \;=\; \beta_0 + \beta_{\text{sf}} \cdot \text{sqft} + \beta_{\text{br}} \cdot \text{bedrooms} + \beta_{\text{w}} \cdot \text{walkability}.$$

This is perfectly sensible; price per square foot is the first thing anyone asks about a housing market. The β's are parameters; `fit` chooses them by minimizing a loss over the data.

Squared loss versus absolute loss

For regression, 0/1 loss is pointless: predicting \$499,999 for a \$500,000 house would be as wrong as predicting \$1. Two natural choices are the squared loss $(y - \hat{y})^2$ (mean squared error) and the absolute loss $|y - \hat{y}|$ (sometimes called L1 loss). How do they differ? Under absolute loss, ten errors of \$1,000 cost the same as one error of \$10,000. Under squared loss, ten errors of \$1,000 cost $10 \times 10^6 = 10^7$, while one error of \$10,000 costs 10^8: ten times more. Squared loss is saying *do not make big mistakes; small mistakes all day long are fine.* Absolute loss is indifferent to how the total error is distributed. Neither is right; they build models with different characters. (An aside for those who like the calculation: with no features at all, the constant that minimizes squared loss is the mean of the y's, and the constant that minimizes absolute loss is the median.)

Making the model richer: interaction terms

Suppose we fit the linear model, held out a test set, and it does reasonably but we want better. Collecting more data is off the table. What else?

Look at what the linear model *asserts*. Two houses with the same walkability, one with one more square foot: the model says the difference in price is β_{sf}, regardless of what the walkability is. But maybe that is wrong for Austin. Downtown, where walkability is high, an extra square foot might be worth a lot; twenty miles out, 2,600 versus 2,700 square feet might barely register. The linear model cannot express "the value of square footage depends on walkability." Every coefficient is a constant.

The fix is a design decision, not a machine-learning one. Add a new term:

$$\widehat{\text{price}} = \beta_0 + \beta_{\text{sf}} \cdot \text{sqft} + \beta_{\text{br}} \cdot \text{bedrooms} + \beta_{\text{w}} \cdot \text{walkability} + \beta_{\text{sw}} \cdot (\text{sqft} \times \text{walkability}).$$

Regroup the two terms involving square footage:

$$(\beta_{\text{sf}} + \beta_{\text{sw}} \cdot \text{walkability}) \cdot \text{sqft},$$

and now the effective price per square foot is itself a function of walkability. A house with walkability 100 gets a different marginal value for square footage than a house with walkability 0. This is called an *interaction term*, and it came from nowhere but our intuition about the problem. Python would never propose it.

Which model is better? Cross-validation, again

We now have two model families on the same data. The one with the interaction term will always have lower or equal *training* loss, because it strictly contains the other (set $\beta_{\text{sw}} = 0$). So training loss cannot decide. The first letter of the answer is C: compute the cross-validation score of each and pick the better one.

Notice what this buys us. Cross-validation is what makes it *safe* to try richer models. I have an intuition that square footage interacts with walkability; I don't have to know whether the data bear it out, because cross-validation will tell me. And once I trust that tool, I can try terms I have no intuition about — bedrooms squared, square footage squared — and let the data decide. I would not go crazy and try millions of candidate terms, for reasons we will get to (there is a cost to picking the winner from a large field), but within reason, cross-validation lets you explore.

The posted notebook does exactly this on salary versus years of experience: a straight line fits badly, and adding an experience-squared term fits much better. We will walk through it next time.

4.5 Where this fits

In classical machine learning — everything before neural networks and embeddings — this is the name of the game. I spent some years at a computational finance firm, and everyone there had access to the same data. The difference between one team and another was exactly this: the intuition that a particular interaction might matter, and the discipline to test it. Domain expertise and cleverness, married to tools that check whether the cleverness is real. The tools are what this class teaches. The intuition you bring.

Lecture 5

Feature engineering, soft scores, and maximum likelihood

Tuesday, September 8, 2026

Questions of the Day

Q1. True or False: In K-fold cross-validation, every data point is used for validation exactly once and for training $K-1$ times.

Q2. True or False: Increasing K makes K-fold cross-validation computationally cheaper because each validation fold is smaller.

Q3. True or False: Cross-validation can help choose between model families or hyperparameter values when training loss alone would favor the more flexible option.

Q4. Free answer: Two models have similar average cross-validation scores, but one model's scores vary dramatically across folds. What does that variation suggest, and why should it make us cautious about that model?

Q5. True or False: Under squared loss, one prediction error that is ten times as large contributes the same total loss as ten errors of the original size.

Q6. True or False: Adding a square-footage-times-walkability interaction feature allows a linear regression model to assign a different marginal value to square footage at different levels of walkability.

5.1 Discussion of the questions

Q1. True. Before answering, the class reconstructed what cross-validation is for, from the ground up. Why split into train and test at all? Because training error cannot tell us which of two families to use; the more flexible one always wins. Why cross-validate rather than split once? Because a single split can be unlucky, and averaging over many splits reduces that. Mechanically: cut the data into K pieces; for each piece, train on the other $K-1$ and validate on it. So each point is validated on once and trained on $K-1$ times.

Q2. False. Each validation fold is indeed smaller, but you do K fits, and fitting is what costs. As a rule of thumb, `model.fit` is expensive and `model.predict` (also called *inference*) is cheap. This will matter even more with neural networks, where training also uses at least twice the memory of inference, and memory is what limits us on the GPUs we have.

Q3. True. This question hides a piece of jargon worth pinning down. A **parameter** is anything `fit` chooses: for a tree, which features to split on and at what thresholds; for a neural network, the weights. A **hyperparameter** is anything you choose before `fit` runs: the depth of the tree, the loss function, the number of layers, the kernel size. The difference is not cosmetic. Parameters are chosen

to minimize training loss. You cannot let hyperparameters be chosen the same way, or you would always get the deepest tree. So hyperparameters are chosen by cross-validation, and then `fit` chooses the parameters for the fixed hyperparameters. When we move to neural networks, `fit` gets called *training*, but it is the same thing. (`sklearn` reflects this: `DecisionTreeClassifier(max_depth=3)` takes hyperparameters as arguments, with sensible defaults if you omit them.)

Q4. Each fold is a small simulation of "train on data like this, test on new data like this." If the results swing wildly across folds, the model's performance depends heavily on which particular data it saw, and you should expect the same volatility on tomorrow's batch. The average may be fine, but your confidence in that average should be low.

This led to a question we had not yet discussed: is the cross-validation score an overestimate, an underestimate, or unbiased as a predictor of performance on new data? For a single, fixed model, it is roughly unbiased. But that is not how we use it. We compute CV scores for 30 candidate models and pick the winner, and the winner's score is optimistically biased. Think of sprinters: the fastest time in a heat was probably also that runner's best race. Picking the winner out of a field picks up some luck along with the skill. So the CV score of the model you selected will typically overstate how it does tomorrow. (This became Q5 of the next class.)

Q5. False, the opposite. Under squared loss, one error of size 10 costs 100; ten errors of size 1 cost 10. Squared loss would much rather make 100 errors of size 1 than one error of size 10. The loss that treats those two cases identically is absolute loss. Neither is better in the abstract; the question is what you want your model to avoid. Some applications tolerate a few big misses in exchange for being very good most of the time. Squared loss says no to that.

Q6. True. The model $\beta_{\text{sf}} \cdot \text{sqft} + \beta_{\text{w}} \cdot \text{walkability}$ forces the value of one extra square foot to be β_{sf} regardless of walkability. Adding a column equal to sqft $\times$ walkability with its own coefficient makes the effective price per square foot $\beta_{\text{sf}} + \beta_{\text{sw}} \cdot \text{walkability}$. The model is no longer linear in the features, but it is still linear in the *parameters*, and that is what "linear regression" means.

5.2 Nonlinear fits with linear regression: the notebook

We walked through the salary-versus-experience notebook. First, the mundane part, which is where most of your time will go in practice. Reading the CSV pulled in an index column as if it were data, so I dropped it. The salary column came in as *strings* (`'71'` rather than `71`), which produced a nonsense scatter plot; the giveaway was quotation marks when I printed the array. `sklearn` wants X as a two-dimensional array of shape $(n, 1)$, not a one-dimensional array of shape $(n,)$, hence the `reshape`. None of this is conceptually hard. All of it will happen to you.

Once the data were clean, the scatter plot showed a clear relationship that is clearly not a line: salary rises with experience, levels off, and eventually declines. `LinearRegression()` with no arguments means "all lines, squared loss," and the best line is a poor fit. Its slope was low (about 0.88) because a steeper line matching the early years would incur huge squared errors on the late-career points, and squared loss refuses to allow that.

What is a better family? We need something that can bend. Try

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2.$$

Is this linear? Yes and no. It is not linear in x. It is linear in $\beta_0, \beta_1, \beta_2$, which are what `fit` searches over, and that is all that matters. The mechanics are exactly what we would do if we had two

features: build a new X with two columns, the first equal to experience and the second equal to experience squared, and call `LinearRegression().fit(X_new, y)` unchanged. Squaring a column in NumPy squares each entry. The same recipe gives you x^3, $\sin x$, or sqft $\times$ walkability: make the column, run the same code.

The fitted coefficients were roughly $\beta_1 \approx 2.87$ and $\beta_2 \approx -0.05$. Read them: early in a career, each year adds nearly 3 units of salary and the quadratic term is negligible; at $x = 10$ the quadratic term subtracts $0.05 \times 100 = 5$ and is starting to bend the curve down. Interpreting the fitted model is not optional. At some point you have to explain what your model is saying, and here you can.

Where did the square come from? Here, from looking at the plot. In the housing example, from intuition about the market. But suppose you are not sure whether mortgage rate times walkability belongs in the model. You don't have to be sure: add the column, and compare CV scores with and without it. Cross-validation is the algorithmic substitute for domain expertise when you lack it, and the check on your domain expertise when you have it.

5.3 Soft scores: from regression to probabilities

Back to the thought from last week. For heart-disease risk from blood pressure and BMI, we do not want "high" or "low"; we want a probability. There are two reasons, one about modeling and one technical.

The modeling reason is obvious: a probability is more useful to a doctor and more useful for deciding how much effort to spend on a customer.

The technical reason will run through the rest of the course. Linear regression — multiply features by coefficients and add — is one of the most flexible and computationally convenient things we know how to do. Neural networks, as we will see, are essentially many copies of exactly this gadget stacked on top of one another. But adding and multiplying numbers will never keep the output between 0 and 1. If I compute a score

$$z = \beta_0 + \beta_1 \cdot \mathrm{BP} + \beta_2 \cdot \mathrm{BMI},$$

it lives anywhere on the real line.

The **sigmoid** (in one dimension; the multi-class version is called **softmax**, and the two names get used interchangeably) solves both problems at once:

$$\sigma(z) \;=\; \frac{e^z}{1+e^z} \;=\; \frac{1}{1+e^{-z}}.$$

Since $e^z > 0$, the ratio is strictly between 0 and 1. When z is large and positive, e^z dominates the 1 and the ratio is essentially 1. When z is large and negative, e^z is essentially 0 and so is the ratio. And $\sigma(0) = 1/2$: a score of zero means "I have no idea."

Just as important, the sigmoid *preserves order*: if $z_1 < z_2$ then $\sigma(z_1) < \sigma(z_2)$. This matters because order is all the linear score carries. Regression says "more square feet, higher score"; the sigmoid must not scramble that. Contrast an alternative that also lands in $[0, 1]$: $z^2/(1+z^2)$. It folds negatives onto positives, so $z = -3$ and $z = +3$ map to the same value, and $z = -3$ maps *above* $z = -1$. That destroys what the regression computed. (This became Q4 of the next class.)

Note that the sigmoid has no parameters. All the parameters are in z. The sigmoid is a fixed, monotone rescaling that exists for our interpretation: it is easier to hear "65% chance of heart disease" than "your z-score is 0.62." The score z is called the **logit**.

In `sklearn`: `model.predict(X)` returns hard labels, which for logistic regression means thresholding the probability at 0.5, equivalently checking whether $z > 0$. `model.predict_proba(X)` returns the soft scores.

5.4 Where do loss functions come from? Maximum likelihood

We now have a catalog of losses: for classification, 0/1, hinge, exponential, and cross-entropy; for regression, squared and absolute. I have told you that changing the loss changes the solution, and that no loss is universally best. Cross-validation cannot choose among them, since you would be evaluating with the same loss you trained with. So where do these come from, and is there any principle behind them? For two of them, cross-entropy and squared loss, there is, and it is worth seeing.

A coin

Flip a coin 100 times: 37 heads, 63 tails. What is the probability of heads? The class offered 37%, and also 50%. Both are reasonable, and they represent two worldviews. Saying 50% brings in prior knowledge: coins are minted to be fair, this one looks normal, 37 out of 100 is a fluctuation. That is the *Bayesian* instinct, and to see why it is not silly, imagine flipping once, getting heads, and declaring the probability of heads to be 100%. Saying 37% uses nothing but the experiment in front of you, plus the single assumption that the flips are independent. That is the *frequentist* instinct. Statisticians argue about these; machine learning people mostly don't, because we do whatever works. Today we follow the second idea.

The principle of maximum likelihood

Suppose the true probability of heads is p. Then the probability of seeing exactly 37 heads in 100 flips is

$$f(p) \;=\; \binom{100}{37} p^{37}(1-p)^{63}.$$

If $p = 1$, this is zero: you would never see a tail. If $p = 0$, also zero. Those extremes are ruled out. Between them, is $p = 0.4$ or $p = 0.5$ more plausible? Compute $f(0.4)$ and $f(0.5)$ and see which makes what we observed more likely. The **principle of maximum likelihood** says: the estimate $\hat{p}$ is the value of p that maximizes $f(p)$, the probability of the data we actually saw.

Finding the maximum is calculus: set the derivative to zero. By the product rule,

$$f'(p) \;\propto\; 37\,p^{36}(1-p)^{63} \;-\; 63\,p^{37}(1-p)^{62} \;=\; p^{36}(1-p)^{62}\,\big[\,37(1-p) - 63p\,\big].$$

The factors p^{36} and $(1-p)^{62}$ vanish only at $p = 0$ or 1, which are the minima. The bracket vanishes when $37 - 100p = 0$, so

$$\hat{p} \;=\; \frac{37}{100}.$$

Our first instinct, 37%, was maximum likelihood all along.

Toward logistic regression

Now the connection. Logistic regression wants to model $P(Y = 1 \mid X = x)$, the probability that a patient with these measurements develops heart disease. Imagine 100 patients with exactly my blood pressure and BMI, of whom 37 developed heart disease. What should the model output for someone like me? By the same reasoning, 37%. Logistic regression is flipping a different coin for every value of x, and we want to apply maximum likelihood to all of them simultaneously. The idea also extends beyond two classes, to CIFAR-10's ten labels or ImageNet's thousand.

The punchline, to be shown next time: cross-entropy loss *is* the maximum likelihood principle applied to the logistic regression model. Minimizing cross-entropy is choosing the parameters that make the observed labels most probable.

Lecture 6

Logistic regression, cross-entropy, and the first look at regularization

Thursday, September 10, 2026

Questions of the Day

Q1. True or False: In a decision tree, maximum depth is a parameter learned by `model.fit`, while the split features and thresholds are hyperparameters chosen by the researcher.

Q2. True or False: A regression model containing X and X^2 can still be fit with linear regression because it is linear in its learned coefficients.

Q3. True or False: The sigmoid function maps every real-valued score to a value between 0 and 1 while preserving the ordering of the scores.

Q4. True or False: Replacing the sigmoid with $Z^2/(1+Z^2)$ would preserve the ordering of all real-valued scores.

Q5. Free answer: A researcher compares 30 models using cross-validation, selects the model with the best score, and reports that winning score as its expected deployment performance. Why might this estimate be overly optimistic?

Q6. Free answer: For 0 heads in 2 independent coin flips, the maximum-likelihood estimate of the probability of heads is ?

6.1 Discussion of the questions

Q1. False; it is the other way around. Parameters are whatever training chooses, and training does one thing: minimize training loss. It does not care about overfitting or anything else. Depth is set by you before training and is a hyperparameter.

This opened a longer discussion that is worth recording in full, because it organizes everything we do about model selection.

How do we detect overfitting? By the gap between training and test performance. If training accuracy is much higher than test accuracy, the model has fit something in the training data that is not in the test data. A gap is not automatically fatal — if test accuracy is high enough, you may be content — but it is the diagnostic.

How do we detect underfitting? Not by low test accuracy; test accuracy can be low for other reasons. Underfitting is revealed by low *training* accuracy. The model was shown the data, allowed to fit it as well as it could, and still could not. That means the family is not rich enough.

Can a model do both? Yes: mediocre training accuracy and much worse test accuracy. Ultimately we want to push test accuracy up, and low test accuracy can come from either problem or both, so

the diagnosis matters.

What do we do about it? With the tools we had before today, only one crude thing. If overfitting, use a simpler model: a shallower tree, fewer interaction terms. If underfitting, a richer one. Shrinking the training set does not help. Getting more data helps but is usually not available. A student suggested manipulating the training data (adding noise, for instance); that is a real technique and we will come back to it. The main new tool, introduced at the end of today, is **regularization**.

Q2. True. A parabola $\beta_0 + \beta_1 x + \beta_2 x^2$ is linear in $\beta_0, \beta_1, \beta_2$, and those are what `fit` chooses. Any feature you can compute from the data — x^2, $\cos^2 x$, $x_1 x_2$ — can be added as a column and fit with ordinary linear regression. What you *cannot* do this way is something like $\beta_1 x + \beta_2^2 x'$, which is nonlinear in a parameter and needs different code. Note also that making a new column is not the same as collecting new data; no one went back to the world to measure anything.

Q3. True. Both properties, and both are why we use it. Between 0 and 1 gives us an interpretable probability. Order-preserving means the sigmoid does not undo what the linear score computed. The point bears repeating: multiplying and adding numbers is one of the most powerful things we know how to do, neural networks are made of nothing else, and the sigmoid is the fixed rescaling that lets us keep using those tools while producing a probability.

Q4. False. Take $z = -3$ and $z = -1$. The first maps to $9/10$, the second to $1/2$: the order is reversed. Squaring folds the negative half of the line onto the positive half, so -3 and $+3$ become indistinguishable. Regression puts information into the sign and magnitude of z, and this map throws the sign away. (We could replace e^z by other increasing functions — later we will see $e^{z/T}$, where T is a *temperature*, the same temperature you can set when using an LLM.)

Q5. Because the reported score belongs to the *winner* of a comparison, and winners are lucky as well as good. For a single fixed model, the CV score is roughly an unbiased estimate of tomorrow's performance. But take ten fair quarters, flip each twenty times, and pick the one with the most heads: it will have shown more than 50% heads, even though every coin is fair. Selecting the best of 30 models does the same thing. There is nothing wrong with the procedure — picking the best by cross-validation is the right thing to do — but the winning score overstates what you will see when you deploy. The phenomenon goes by several names: winner's curse, selection bias, survivor bias (studying only the companies that made it big). The stock that has done best to date is probably not going to do quite that well tomorrow.

Q6. Zero. Two flips, two tails: the value of p that makes "tails, tails" most probable is $p = 0$. This is an extreme example chosen on purpose. Maximum likelihood is a principle that tells you how a reasonable estimate can be derived, and it makes sense most of the time, but a principle is not a guarantee. With two flips, the Bayesian who says "coins are usually fair" has a point.

6.2 Logistic regression: the model

Recall the setup. We have d features (blood pressure, cholesterol, BMI, ...) and want $P(Y = 1 \mid X = x)$. Step one, compute a linear score, the *logit*:

$$z(x) \;=\; \beta_0 + \beta_1 x_1 + \cdots + \beta_d x_d \;=\; \beta_0 + \beta^\top x.$$

(The dot product $\beta^\top x$ is just shorthand for the sum of products, coordinate by coordinate.) Step two, pass it through the sigmoid:

$$\hat{p}(x) \;=\; \sigma(z(x)) \;=\; \frac{e^{z(x)}}{1+e^{z(x)}}.$$

The parameters β_0, β come from `fit`. Hold that off for a moment and ask what a given set of parameters *says*.

The decision boundary is a line

The **decision boundary** is where the model has no opinion: $\hat{p} = 1/2$. Since $\sigma(z) = 1/2$ exactly when $z = 0$, the boundary is the set of x with

$$\beta_0 + \beta_1 x_1 + \beta_2 x_2 \;=\; 0.$$

In two features, that is the equation of a line. Always. For $\beta = (0, 2, -2)$ it is $x_1 = x_2$, the 45° diagonal. For $\beta = (-5, 1, 0)$ it is $x_1 = 5$, a vertical line. For $\beta = (-1, 0, 1)$ it is $x_2 = 1$. Whatever the coefficients, a line; in more dimensions, a plane or hyperplane. Recall that decision trees could only draw axis-aligned cuts; logistic regression draws lines at any angle, but only lines.

I am spending time on this because it is the seed of the most important idea of the semester. If logistic regression gets low error on a dataset, you have learned that the classes are *linearly separable*, or nearly so. What we will see when we get to neural networks is that ResNet-18, ten million parameters, takes in pixel intensities and *transforms* them into a space where the classes are linearly separable, and the last layer of the network is logistic regression. That sentence contains the whole class, and we will unpack it slowly.

The boundary is scale-invariant; the model is not

Compare $\beta = (-5, 1, 0)$, $(-10, 2, 0)$, and $(-15, 3, 0)$. Multiply the equation $\beta_0 + \beta_1 x_1 + \beta_2 x_2 = 0$ by any positive constant and the solutions do not change, so all three have the same boundary, $x_1 = 5$. `model.predict` gives identical answers for all three; their accuracy is identical.

But they are different models. Take a point just barely on the positive side, with logit $z = 0.1$ under the first model. Then $\hat{p} = \sigma(0.1) \approx 0.52$: barely blue, almost a coin toss. Under the model with coefficients five times larger, the same point has $z = 0.5$ and $\hat{p} \approx 0.62$. Multiply by 50 and $z = 5$, $\hat{p} \approx 0.993$. Scaling the coefficients up makes the model more *confident* about every point without moving the boundary; scaling toward zero makes it less confident, until at scale zero it says 50–50 about everything. `predict` cannot see this; `predict_proba` can.

Now the question: on a dataset that is perfectly separated by the line $x_1 + x_2 = 10$, which model is better, $z = -10 + x_1 + x_2$ or $z = -50 + 5x_1 + 5x_2$? Both are 100% accurate. Intuitively the second, because it is correct *and* sure of it. But then $z = -500 + 50x_1 + 50x_2$ is better still, and so on without end. In this example there is no best model. Something has gone wrong, and to say what, we need to stop saying "better" intuitively and write down the loss.

6.3 Where cross-entropy comes from

Consider five data points with labels $y = (1, 0, 1, 1, 0)$ and two models.

true label y	1	0	1	1	0
Model A: $\hat{p}$	0.60	0.40	0.55	0.52	0.45
Model B: $\hat{p}$	0.90	0.55	0.45	0.85	0.10

Threshold at 0.5 and Model A gets all five right; Model B gets three of five. On accuracy, A wins. But look at what A is doing: it is barely committing to anything. Every answer is "I really don't know, but if forced…" B is wrong twice, but where it is right, it is confidently right. If a student answered every question with "I don't know, but maybe…" and happened to be right, I would not leave thinking she understood the material. I would prefer the student who clearly knows some things and clearly needs to work on others. Accuracy has no way to express this preference. Likelihood does.

The likelihood of a model

For one coin flip with probability p of heads, the probability of the outcome is p if heads and $1-p$ if tails. Writing $y = 1$ for heads, there is a compact way to say this:

$$P(\text{outcome } y) \;=\; p^{\,y}(1-p)^{\,1-y},$$

because when $y = 1$ the second factor is $(1-p)^0 = 1$ and when $y = 0$ the first factor is $p^0 = 1$. It is only notation, but it lets us write one formula instead of two cases.

For logistic regression, each data point x_i has its own coin with $p_i = \sigma(z(x_i))$, and independent events multiply, so the probability of observing all the labels we saw is

$$\mathcal{L}(\beta_0, \beta) \;=\; \prod_{i=1}^{n} \sigma\big(z(x_i)\big)^{\,y_i} \Big(1 - \sigma\big(z(x_i)\big)\Big)^{\,1-y_i}.$$

This is the **likelihood** of the model. For the two models above, $\mathcal{L}_A \approx 0.057$ and $\mathcal{L}_B \approx 0.139$, more than twice as large. Likelihood prefers B, for exactly the reason we wanted: it rewards confident correct predictions and punishes confident wrong ones, and A's timidity costs it on every single point. Maximum likelihood says: choose the parameters that make $\mathcal{L}$ as large as possible.

From likelihood to loss

Products are awkward; we want a sum over data points, because that is the form every loss has had. Take the logarithm, which turns products into sums and brings exponents down. Maximizing $\mathcal{L}$ is the same as maximizing $\log \mathcal{L}$. Then put a minus sign in front, because probabilities lie in $[0, 1]$ and their logs are negative, and we want a loss that is positive and small when the model is good. The result is the **negative log-likelihood**:

$$-\log \mathcal{L} \;=\; \sum_{i=1}^{n} \Big[-\; y_i \log \sigma\big(z(x_i)\big) \;-\; (1-y_i) \log \Big(1 - \sigma\big(z(x_i)\big)\Big)\Big].$$

Read one term. If $y_i = 1$, only $-\log \sigma(z_i)$ survives; it is near zero when $\sigma(z_i)$ is near 1 (confident, correct) and blows up as $\sigma(z_i) \to 0$ (confident, wrong). If $y_i = 0$, only $-\log(1 - \sigma(z_i))$ survives and the same story holds with the roles reversed. Small loss means the model assigns high probability to the labels that actually occurred.

This expression is the **cross-entropy loss**. It is what `model.fit` minimizes when you run `LogisticRegression`, and it is what we will use for nearly every classification network this semester. It was not pulled from a hat: it is the maximum likelihood principle, applied to a model that flips a sigmoid-coin for every data point. (The same argument applied to a regression model with Gaussian noise yields squared loss; that is where mean squared error comes from.)

6.4 The problem with separable data, and the idea of regularization

Now we can return to the question with the right vocabulary. When the data are perfectly linearly separable, what does minimizing cross-entropy do? Every correctly classified point's loss shrinks toward zero as its $|z_i|$ grows. Scaling all the coefficients up makes every $|z_i|$ larger without misclassifying anyone. So the loss keeps decreasing, and `fit` will push the coefficients toward infinity. There is no minimizer.

A more pointed example: many red points stacked in a vertical column, many blue points far to the right, and one blue point just to the right of the red column. Where should a vertical boundary go? Class offered several sensible answers: halfway between the clusters; a bit left of halfway, pulled by the lone blue point; right up against the red column, since that is what gets zero training error. Unregularized logistic regression gives none of them. It puts the line right against the red column so that everything is classified correctly, then scales the coefficients up until it is 99.99% certain about red points sitting a hair from the boundary. To the eye, that is a strange solution.

Here is the discipline I want us to keep. When an algorithm returns something that offends your intuition, do not override it by hand and draw the line where you like. That is shooting from the hip. The right conclusion is that you have not *encoded* your intuition in what you asked the algorithm to do. So encode it.

What went wrong is that the coefficients were free to grow without bound. Stop that. Add a penalty on their size:

$$\min_{\beta_0,\beta} \; \sum_{i=1}^{n} \mathrm{CE}(y_i, \hat{p}_i) \; + \; \lambda \, (\beta_0^2 + \beta_1^2 + \cdots + \beta_d^2).$$

This is **regularization**. The first term is the loss, which wants big coefficients. The second term knows nothing about the data — no x, no y — and simply grows with the coefficients. `fit` must now balance the two: achieve low loss, but do not use extreme weights. The constant λ sets the balance. With λ near zero we recover the boundary jammed against the red column with huge coefficients. As λ increases, the coefficients that produced that steep, over-certain profile become too expensive, and the boundary moves toward the middle and becomes less certain. Every one of the "sensible" answers from class is available for some λ, chosen in a principled way rather than by hand.

Two remarks. First, a technicality: `sklearn`'s `LogisticRegression` exposes the regularization strength as a parameter C with $\lambda = 1/C$. So *large* C means weak regularization (I set $C = 10^6$ to turn it essentially off) and small C means strong. (`Ridge` for linear regression uses the constant directly.) Second, and more important: I asked at the start of class what tools we have against overfitting, and the only answer was "use a simpler model" — go from depth 3 to depth 2, delete the interaction term. That is a hard, manual way to restrict the family. Regularization is a soft, algorithmic way to do the same thing. The parameter space is unchanged; large coefficients are still allowed; they just cost something. We are still shrinking the effective set of models we search over, but continuously, with a knob. Next time we see what this knob does in practice.

Homework 1 will be posted tomorrow: Python exercises on these topics, with CIFAR-10.

Lecture 7

Diagnosing fit, ridge regression, and the fitted model as a random variable

Tuesday, September 15, 2026

Questions of the Day

Q1. True or False: A model with high training error and substantially higher testing error can be both underfitting and overfitting.

Q2. True or False: For binary logistic regression, the decision boundary occurs where the predicted probability is 0.5, equivalently where the logit Z equals 0.

Q3. True or False: Multiplying every coefficient of a logistic regression model by the same positive constant changes its decision boundary but leaves its predicted probabilities unchanged.

Q4. True or False: Two logistic regression models can have identical hard predictions and accuracy but different cross-entropy losses because their confidence levels differ.

Q5. Free answer: Suppose a logistic regression data set is perfectly linearly separable and the model is trained by minimizing cross-entropy without regularization. What can happen to the coefficient magnitudes, and why?

Q6. True or False: In `sklearn` logistic regression, increasing C strengthens the regularization penalty and pushes the learned coefficients closer to zero.

7.1 Homework 1

Homework 1 is posted; it is due in two weeks (two Tuesdays from today), one submission per group with all members listed. It walks through everything so far: looking at a dataset and guessing what unlabeled features mean; train/test curves for trees on MNIST and then on CIFAR-10 (which is big enough that computation starts to matter, so be judicious about how much data you train on while exploring); a cross-validation experiment on the winner's curse; synthetic housing data with square footage, bedrooms, and walkability, comparing squared and absolute loss when a few houses sit far outside the main range; cross-entropy versus accuracy on the five-point example from class; regularization; and two things we start today, the fitted model as a random variable, and ridge and lasso.

One framing remark. Trees get about 70% on MNIST and about 23–25% on CIFAR-10, which is why CIFAR-10 is our working problem. Getting from 25% to about 80% will be about building richer models. Getting above 90% will require a different idea, transfer learning. That is the arc of the semester.

7.2 Discussion of the questions

Q1. True, and the reasoning is the important part. Low test accuracy by itself does not tell you which problem you have.

Sketch the MNIST curves: accuracy against depth, training in black rising toward 100%, test in red tracking it and then flattening near 70%. At very low depth both curves are low and essentially equal. Are we overfitting there? No. Overfitting means the model has fit something in the training data that is absent from the test data, and the evidence for that is a *gap* between the curves. No gap, no overfitting. Yet accuracy is terrible. That is **underfitting**: a property of the black curve alone. The model saw the data and could not fit it; it lacks the flexibility. At high depth the gap is wide: **overfitting**.

Everything here is algorithmic. `fit`, then `predict` on the training X (black curve) and on the test X (red curve). Compare them. No intuition about the problem, no plotting of decision regions.

Why does the diagnosis matter? Because the prescriptions are opposite. Underfitting calls for a *richer* model: a deeper tree; for logistic or linear regression, feature engineering (add interaction terms, squares), or weaker regularization. Overfitting calls for a *simpler* model: a shallower tree, fewer engineered features, or stronger regularization. You can see the same disappointing test accuracy in both regimes and need to do completely different things.

Now sketch what you will see on CIFAR-10 with trees. Both curves start low together (underfitting). The black curve climbs to 100%; the red curve rises a little, then falls back, so that at high depth test accuracy is about where it was at low depth (overfitting). In the middle, the gap is already wide *and* training accuracy is still poor: both at once. That is the answer to the question, and it is also the bad news about trees: there is no depth at which they are neither underfitting nor overfitting on CIFAR-10. What we want is a family whose test curve tracks the training curve to a much higher level before separating. Depth alone will not get us there; we need more knobs.

Q2. True. The class rebuilt this from scratch. Logistic regression is a classifier that outputs soft probabilities. For two classes we report one number, $\hat{p}$ (the other is $1-\hat{p}$); for three classes we would have to report all three. The boundary is where the model expresses no preference: $\hat{p} = 1/2$. With $\hat{p} = e^z/(1+e^z)$ (equivalently $1/(1+e^{-z})$, multiply top and bottom by e^{-z}), that happens exactly when $z = 0$. In two features, $z = \beta_0 + \beta_1 x_1 + \beta_2 x_2$, so the boundary is the line $\beta_0 + \beta_1 x_1 + \beta_2 x_2 = 0$. The parameters determine the line; `fit` determines the parameters by minimizing cross-entropy.

Two side points. First, a reminder about features: the triangles-and-circles picture has two features, x_1 and x_2, whatever the number of classes. MNIST has 784 features; CIFAR-10 has 3,072. Second, with more than two classes logistic regression still partitions the space with straight boundaries, and the last step of *every* neural network — classifying images, generating language — is logistic regression. Old-fashioned as it sounds, facts about when logistic regression can and cannot succeed are facts about neural networks.

A challenge problem: the ring

Here is a tangent I could not resist. Binary classification, two features. The circles cluster around the origin; the squares form a ring around them at larger radius. The signal is obviously there. What does `LogisticRegression().fit(X, y)` do? Badly. Its boundary is a line, and no line separates a disk from the ring around it; it will find some line that does slightly better than chance and stop.

You are allowed to use only `LogisticRegression` and `fit`. How do you solve it? Feature engineering. Add a third feature

$$x_3 \;=\; x_1^2 + x_2^2,$$

the squared distance from the origin. Now the circles have small x_3 and the squares have large x_3, and a boundary of the form $\beta_0 + \beta_3 x_3 = 0$ separates them perfectly. Drawn in the original (x_1, x_2) plane, that boundary is a circle. Have we contradicted "logistic regression is always linear"? No. It is linear in the *three*-dimensional feature space; the circle is what a plane in (x_1, x_2, x_3) looks like when projected onto (x_1, x_2). In fact we could throw away x_1 and x_2 and keep only x_3: on that one-dimensional line, all the circles pile up near zero and all the squares pile up farther out, and the problem is trivially linearly separable.

This stylized story is a preview of what neural networks do. Cats and dogs and boats in CIFAR-10 are hopelessly tangled in 3,072-dimensional pixel space. The first $N-1$ layers of a network perform a nonlinear transformation of the data — learned from data rather than designed by hand, but the same kind of thing as $x_1^2 + x_2^2$ — into a space where the classes are linearly separable, and the last layer is logistic regression. The transformed space is where "dog space" and "cat space" become geometric regions, and that is the doorway to generative AI. We will come back to this.

Q3. False; it is exactly reversed. Multiply $\beta_0, \beta_1, \beta_2$ by 10: the equation $z = 0$ has the same solutions, so the boundary does not move. But every z is ten times larger, so every probability moves away from 1/2: a point that was 80% red becomes more certain; the model becomes more opinionated everywhere. Multiply by a constant less than 1 and the probabilities move toward 1/2; multiply by something tiny and the model has no opinion about anything, though its boundary is still exactly where it was.

Q4. True. Identical hard predictions means the same boundary, which is what `predict` reports. Cross-entropy depends on the probabilities, which `predict_proba` reports, and those differ with scale. This is precisely the situation of Q3.

Q5. The coefficients grow without bound. Unregularized, the objective is only cross-entropy, and cross-entropy on a correctly classified point keeps shrinking as $|z|$ grows. On separable data every point can be made correctly classified, so scaling all coefficients up reduces the loss forever. We saw the numbers last time: the same point at 52%, then 62%, then 99.9% as we multiplied by 5 and then by 50. The model ends up saying that the instant you step across the line you are 100% a triangle, and the instant you step back you are 100% a square. Regularization penalizes coefficient size and stops this. That is why we need it.

Q6. False, but only because of the first half. In `sklearn`'s logistic regression the strength of the regularization is $\lambda = 1/C$, so *increasing* C weakens the penalty. The second half is correct: stronger regularization (larger λ, smaller C) pushes the coefficients toward zero. (For reasons unknown to me, `Ridge` in the same library uses the multiplier directly.)

7.3 Regularization, once more

The regularized problem is

$$\min_{\theta} \;\; \sum_{i=1}^{n} \text{Loss}\big(y_i,\, h_\theta(x_i)\big) \;+\; \lambda\, R(\theta),$$

where R penalizes the size of the parameters, most commonly the sum of their squares. The loss term looks at the data. The penalty term does not; it is an anchor pulling the solution toward zero.

Larger λ pulls harder. This is a general and powerful idea, not specific to logistic regression: it applies to any model and any loss, and it is a *continuous* way to make a model simpler. Rather than deleting the square-footage-times-walkability column because the model overfit, keep it and let the penalty decide how much of it to use.

Ridge regression

Apply this to linear regression with squared loss and a squared penalty:

$$\min_{\beta} \; \sum_{i=1}^{n} \left(y_i - x_i^\top \beta\right)^2 \; + \; \lambda \|\beta\|^2.$$

This is called **ridge regression**, also L2 regularization, also (in the neural network world) *weight decay*. In one dimension you can solve it by taking the derivative and setting it to zero:

$$-2\sum_i x_i \left(y_i - x_i\beta\right) + 2\lambda\beta = 0 \quad \Longrightarrow \quad \hat{\beta}_{\text{ridge}} \; = \; \frac{\sum_i x_i y_i}{\sum_i x_i^2 + \lambda}.$$

(In matrix form, $\hat{\beta} = (X^\top X + \lambda I)^{-1} X^\top y$; the ordinary least-squares solution is the case $\lambda = 0$.)

Look at the λ in the denominator. As $\lambda \to \infty$, $\hat{\beta} \to 0$. What is the overfitting of the model $\hat{\beta} = 0$? Zero. It does not look at the training data at all, so it does equally terribly on training and test, and the gap between them is nothing. Nobody would deploy that model, but it tells you the knob works: at one extreme you have unregularized least squares with whatever overfitting that entails, at the other you have a model that cannot overfit, and in between you have a continuum. The hope is that a moderate λ tempers overfitting without destroying the fit. Whether it does, and how much, is what cross-validation over λ will tell you.

7.4 The fitted model is a random variable

The word "variance" has been lurking. Here is what it means for us.

`LinearRegression().fit(X, y)` is deterministic. Give it the same data twice and you get the same $\hat{\beta}$ (read off as `model.coef_`) both times. Nevertheless, $\hat{\beta}$ is a random variable. Where does the randomness come from? From the data. Collect a hundred days of Coke and Pepsi stock prices and regress one on the other; collect a *different* hundred days and do it again. The two slopes will be close, but they will not be equal. Every batch of data you could have collected yields a slightly different fitted model, and the spread across those possible batches is the variance of the fitted model.

Why should you care? Because tomorrow's data are another such batch. The variance of $\hat{\beta}$ across batches is a statement about how far the model you happened to fit might be from the model you would have fit on slightly different data, which is a statement about reliability.

A synthetic experiment

To see this, play nature. Generate x from a Gaussian; generate $y = 15x +$ noise, with the noise also Gaussian. Make a thousand such points and run linear regression. What is $\hat{\beta}$? The class said 15, and that is right on average, but it will never be exactly 15.000, because of the noise, and it gets closer to 15 as you generate more points. (Think of the extreme: with a single point, you could be

far off.) Even in this problem, where the model family is exactly right and we are on home turf, the fitted coefficient is not the true coefficient.

Now repeat the whole thing thousands of times and histogram the $\hat{\beta}$'s. With no regularization you get a bell curve centered at 15 with some spread: sometimes the noise happened to lean positive and the slope looked bigger than 15, sometimes the reverse. The estimate is **unbiased** — right on average — but variable.

Now regularize, and increase λ. Two things happen to the histogram. Its spread *shrinks*. And its center *shifts* toward zero: at high regularization it is centered around 8, not 15. The estimate is now **biased** — wrong on average — but much less variable.

This is the **bias–variance trade-off**, a phrase you have all heard and can now attach to a picture. Why would anyone prefer the biased estimator at the bottom right of that figure? Because its spread is so much smaller: the model you get from your particular batch of data is much closer to the model you would get from any other batch. We pick this up on Thursday, together with why this is another way to understand overfitting and underfitting. The notebook is posted if you want to run it before then; you do not need it to start the homework.

Lecture 8

Bias, variance, ridge, lasso, and the road to ensembling

Thursday, September 17, 2026

Questions of the Day

Q1. True or False: Logistic regression cannot separate points near the origin from points in a ring using only X_1 and X_2, but it can do so after appropriate feature engineering.

Q2. True or False: Making the ridge regularization strength extremely large can eliminate overfitting while still causing severe underfitting.

Q3. True or False: The ridge regularization penalty depends directly on the observed X and Y values, so difficult training examples receive a larger penalty.

Q4. True or False: Even when `model.fit` is deterministic, its fitted coefficients can be random variables because a different random sample of training data would produce different coefficients.

Q5. Free answer: Repeated samples from the same stochastic (random) process produce unregularized slope estimates centered near 15 with a wide spread. After regularization, the estimates are centered near 8 with a much narrower spread. Explain the trade-off and why the regularized estimator might still be useful.

Q6. True or False: Increasing ridge regularization generally reduces the bias of the fitted coefficients while increasing their variance.

8.1 Discussion of the questions

Q1. True, and a lot is packed into it. First, logistic regression is a linear method: its decision boundary is the set of points where the score z is zero, and in the original features that set is a line. Second, logistic regression is more than its boundary: at the boundary it says 50–50, and how fast it becomes certain as you move away is governed by the size of the coefficients. Third, if you add new features that are *nonlinear* in the original ones — $x_1^2 + x_2^2$ for the ring — the boundary is still linear in the new, larger feature space, but drawn back in the original plane it is curved. That is the whole trick. In a sense every neural network is some feature engineering followed by one logistic regression step, which is why we keep returning to this.

Q2. True. Drive λ to infinity and every coefficient goes to zero. A model with all-zero coefficients does not look at the data, so its answer cannot depend on the training set, so it cannot overfit. It also cannot fit anything: its training error is terrible, which is underfitting. Test error comes from two sources, overfitting and underfitting, and they can be present at the same time. Eliminating one by destroying the model is not a solution.

Q3. False, and this is the point of the design. The penalty depends only on the coefficients; there is no x and no y in it, and no data point is treated differently from any other. That sounds backwards at first — shouldn't a penalty know about the data? — but training already does that: it says "here are the points, fit them as closely as you can." Regularization gives the optimizer something else to juggle that has nothing to do with the data, precisely so that it cannot chase every last detail of the training set.

Q4. True. Everything in `sklearn` that we have used is deterministic: same X and y, same $\hat{\beta}$, no random seed needed. (Training a neural network will not be like this; there is an internal source of randomness, which we will get to.) But the data are one sample from whatever process generates them. Collect a second batch from the same process and you get a different $\hat{\beta}$.

Why does this matter? Suppose we study how grades relate to hours of sleep, in this class, and find that beyond six hours each additional hour is positively correlated with grades. (Correlated; I am not claiming anything causal.) If I ran the same study on next year's cohort I would get slightly different numbers. The question is whether they could be so different that the sign flips. If the coefficient is 0.5 ± 0.01, the conclusion stands. If it is 0.5 ± 0.75, we have learned nothing. Every time we collect data and fit a model we are drawing one sample of a random variable, and we would like to know its variance, and ideally reduce it. Regularization is a tool for that, and variance is another way to think about overfitting.

Q5. This is the picture from the end of last class. With no regularization, the histogram of fitted slopes across many batches is a bell centered at the truth, 15, but wide. With regularization it is centered at 8, wrong on average, but narrow. Why would anyone accept the biased one?

Some vocabulary first. Nature has a true slope $\beta^* = 15$, and hands us noisy samples. From each batch we compute $\hat{\beta}$: 15.2 one time, 13.8 another. If you could repeat the experiment forever and average all the $\hat{\beta}$'s, would you get exactly 15? If yes, the estimator is **unbiased**. Ordinary linear regression is unbiased — it does not systematically over- or under-estimate — and that sounds like an unambiguously good property.

Now think in extremes. Suppose the unbiased estimator has a standard deviation of 1,000, so the histogram runs from $-1{,}000$ to $+1{,}000$ and is centered at 15. In real life you do not get to see the histogram. You get *one* draw from it: one batch of a hundred data points, one `fit`. You can't rerun the study until next year's cohort arrives, or the next hundred trading days pass, or another hundred patients are recruited at great expense. A rule of thumb: one draw from a bell curve typically lands about one standard deviation from the center. So a single draw from this histogram tells you essentially nothing about 15; the noise washes it out, unbiased or not.

Now suppose a second estimator is centered at 14.5 with standard deviation 1. It is biased — it systematically underestimates — but a single draw will almost surely land between 13 and 16. For the person who has one batch of data and needs an answer now, that is far more useful. This example is deliberately extreme; the real trade-off is gentler. But it shows why a biased estimator can be the right choice, and why we want a knob that moves along this trade-off.

Q6. False; exactly reversed. Increasing ridge regularization increases bias (the center moves toward zero) and decreases variance (the spread shrinks).

8.2 Variance: a rule of thumb worth knowing

In the ridge notebook we generate $y = -2x + \text{noise}$, fit a line, and get -2.07, then -1.94 on a fresh batch. Two things control how far off we are: the amount of data n and the quality of the data, meaning the size of the noise ϵ. How does the spread of $\hat{\beta}$ shrink as n grows?

I like to reason with coins, because laws that reveal themselves with coins turn out to hold for linear regression and for a huge class of machine-learning models. Flip a fair coin 1,000 times. You expect 500 heads, and you will not get exactly 500. What range should you expect? A thousand heads: essentially impossible. Eight hundred: still not happening. To say how wide the range is you need the standard deviation σ, and then the usual facts apply: about 65% of the time you land within one σ of the mean, about 95% within two, and beyond three is well under 1%. So the practical question is what σ is, and the answer that matters is how it scales:

$$\sigma \;\propto\; \frac{\epsilon}{\sqrt{n}}.$$

I do not care whether the constant in front is 1 or 10. What matters is the shape. To cut the standard deviation in half you need *four* times as much data. If you have 50,000 points and are wondering whether collecting 2,000 more is worth it, $1/\sqrt{50{,}000}$ and $1/\sqrt{52{,}000}$ are essentially the same number, so no. On the other hand σ scales *linearly* in the noise level: halve ϵ (buy a better sensor, say) and you halve σ.

Why bother with this when we could just run the experiment? Because collecting more data and buying better sensors are completely different actions with different prices, and this rule lets you compare them *before* spending the money. It is the same reason you learn arithmetic even though you own a calculator: the exact answer is not the point, a sense of magnitude is. The notebook is set up so you can test the $1/\sqrt{n}$ law yourself: change n, change ϵ, and watch the histogram. (Increasing the number of repetitions only makes the bell smoother; it does not change its width.)

8.3 Ridge regression in action

Ridge regression, also called L2 regularization, also weight decay in the neural-network world, also Tikhonov regularization in older literature (everyone who rediscovers it names it), adds $\lambda\|\beta\|^2$ to the squared loss. A standard visualization from `sklearn` plots all ten coefficients of a ten-feature problem against λ on a log scale from 10^{-10} to essentially infinity: at the left they are whatever least squares says, and as λ increases every one of them slides smoothly toward zero.

A polynomial that should not have been fit

Ten data points generated by $y = -1.8x + \text{noise}$. I fit a degree-6 polynomial to them, by hand-building the augmented matrix whose columns are $x, x^2, \ldots, x^6$ and running plain linear regression on it, as we did for the salary data. No one should do this — six degrees of freedom for ten points — but it makes the phenomenon visible. The fitted curve wiggles through the points, and between them it is nowhere near the true line; a new test point at x has its true y on the line and the model's prediction far above it. Why did `fit` not just set the higher-order coefficients to zero, since the truth is a line? Because it can do better on training error by not doing so. There is noise, it has the flexibility to fit the noise, and its only instruction was to minimize training loss. Overfitting, as expected.

Now add ridge regularization instead of cutting the degree back to one. The coefficient on x^6 goes from about 0.57 to 0.07; the coefficient on x^3 from about 5.6 to 0.5. The coefficient on x hardly moves, and that is the term that should be -1.8. The optimizer was paying a large penalty (5.6^2 and so on) for terms that bought it only the ability to fit noise, and it gave them up; it kept the term that fits the actual trend, because that one earns back its penalty in training error. That is the mechanism. Setting everything to zero would minimize the penalty but ruin the training error; the balance point is a model that mostly finds the trend.

Collinearity

A second, more realistic example in the notebook: *collinear* columns, meaning one feature is well predicted by the others. Given a house's location and square footage, you can probably guess better than chance whether it has a pool; the pool column is not independent of the rest. This is not a defect. Storage and computation are cheap, so of course you collect every piece of information you can about your customers or patients. But near-redundant columns create a problem for plain least squares (the linear algebra is in the notebook if you want it), and the punchline is that ridge regularization handles it well. Work through it on your own.

8.4 Lasso: a different regularizer, a different solution

Replace the squared penalty with an absolute-value penalty:

$$\min_{\beta} \ \sum_{i=1}^{n} \left(y_i - x_i^\top \beta\right)^2 \ + \ \lambda \sum_j |\beta_j|.$$

This is **lasso**, or L1 regularization. The two endpoints are the same as for ridge: $\lambda = 0$ is unregularized least squares, and $\lambda \to \infty$ drives every coefficient to zero, because even a tiny nonzero β_j that improves training error slightly is penalized more than it is worth. What differs is the path between the endpoints.

More features than samples

Consider a regression problem with $n = 100$ samples and 200 features. If you could increase one of those numbers, you would want more samples. Here is why. Unregularized linear regression on this problem achieves training error *exactly zero*. Take a smaller version: four features, so five parameters $\beta_0, \ldots, \beta_4$, and three data points. Fitting the three points perfectly means solving three equations,

$$y^{(k)} = \beta_0 + \beta_1 x_1^{(k)} + \beta_2 x_2^{(k)} + \beta_3 x_3^{(k)} + \beta_4 x_4^{(k)}, \qquad k = 1, 2, 3,$$

in the five unknowns β (the x's are data, not unknowns). Three equations, five unknowns: infinitely many exact solutions. Each data point is a constraint; each feature is a degree of freedom. With more features than samples the model can hit every training point exactly, and that tells you nothing about the truth. If someone hands you such a problem, the honest answer is that it cannot be solved without additional structure.

But often there is additional structure, and it mirrors real life. We measure everything about a patient, but only a few measurements actually predict what is wrong. We collect two hundred attributes of a customer, but there are a few reasons someone buys or doesn't. Life is usually simpler than the data we collect; we just don't know in advance which few features matter. If we knew, we

would not have collected the other 195. The toy version: 200 features, of which only 5 have nonzero true coefficients (all equal to 10), and we do not know which 5.

What the three methods do

Plot the 200 fitted coefficients. Unregularized least squares is useless: coefficients everywhere, no indication of which five matter. Ridge is also useless: it shrinks everything toward zero uniformly, so the picture is the same at a smaller scale, and it still has no idea which five are real. That should not surprise you. Ridge penalizes the square, so a single coefficient of 10 costs 100 while spreading that same weight thinly over many coefficients costs much less; the squared penalty actively prefers many small coefficients over a few large ones. Sweeping λ for ridge does not help; there is no value that works.

Lasso, with an arbitrary first choice of λ, already gives a qualitatively different answer: most of the coefficients are *exactly* zero, not merely small (the notebook prints them so you can see). The absolute-value penalty charges 10 for one coefficient of 10 and also 10 for ten coefficients of 1, so it has no preference for spreading weight around, and it turns out to prefer switching features off entirely. Search over a grid of λ for the best cross-validation score and lasso identifies the five important features unambiguously, with the other 195 set to zero; it does not recover the value 10 exactly, but it finds the structure.

Summary

Ridge and lasso agree at $\lambda = 0$ and at $\lambda = \infty$ and behave completely differently in between. Use **ridge** when you think you are overfitting because of noise or collinearity; it controls variance. Use **lasso** when you think many of your features are irrelevant but you do not know which; it produces sparse solutions. There is a large body of theory about linear regression behind these statements, but what matters for us is that, like the $1/\sqrt{n}$ rule, the phenomena carry over to models where the theory is much harder: people use L1 regularization on neural networks and on boosting. I illustrate on simple problems because there the story is clear. The lesson is not the code or the toy example; it is knowing what tools exist and what each one does.

8.5 Toward ensembling

We closed by looking at the plot you will make in the homework: training and test accuracy of decision trees on CIFAR-10 against depth. Training accuracy climbs steadily to 100%. Test accuracy is bad everywhere. This is the diagram from last time, now on real data.

Shallow trees have a small train–test gap, so they do not overfit, but their training accuracy is low, so they underfit. Deep trees do not underfit — training accuracy is high — but they overfit badly. There is no depth in between where both problems are absent. On MNIST trees at least reached 70%; here there is no good option at all.

This is exactly the situation that leads to **ensembling**, the most important algorithmic idea in the classical half of this course and the thing we have been building toward. Two strategies, both about combining trees. **Bagging** starts from deep trees, keeps what is good about them (flexibility, no underfitting), and finds a way to combine many of them so that the overfitting shrinks without the underfitting coming back; applied to trees this becomes *random forests*. **Boosting** starts from shallow trees, keeps what is good about them (no overfitting), and combines them so that each

corrects the errors of the others, reducing the underfitting. Each strategy takes one end of the depth axis and repairs its weakness. Over the next three lectures or so we will build both, and they will be your friends on Kaggle.

Lecture 9

Bagging, the bootstrap, and random forests

Tuesday, September 22, 2026

Questions of the Day

Q1. True or False: If an estimator's standard deviation scales as $1/\sqrt{n}$ in the sample size, doubling the amount of data cuts its standard deviation in half.

Q2. True or False: When a linear regression has more fitted coefficients than training observations, it may achieve zero training error even though its coefficients do not reliably identify the true relationship.

Q3. True or False: Ridge regression is especially useful for feature selection because it typically sets irrelevant coefficients exactly equal to zero.

Q4. True or False: Ridge and LASSO have the same solutions at zero and infinitely strong regularization, but they can produce very different solutions between those extremes.

Q5. Free answer: A data set has 100 observations and 200 features, but only five features truly matter. Explain why ordinary linear regression can fit the training data perfectly yet be uninformative, and why LASSO may be more useful than ridge regression.

Q6. True or False: When one feature can be predicted well from other features, ridge regularization can make the fitted linear regression coefficients more stable.

9.1 The questions

We had a lot to cover, so these were not discussed in class. They review Lecture 8; short answers follow.

Q1. False. Doubling n multiplies the standard deviation by $1/\sqrt{2} \approx 0.71$. To cut it in half you need four times the data. This is exactly the calculation we do below for averaging.

Q2. True. Each observation is one equation and each coefficient is one unknown. With more unknowns than equations there are infinitely many exact solutions, so training error is zero and the particular solution returned tells you nothing about the truth.

Q3. False. Ridge shrinks every coefficient toward zero but leaves them nonzero; it does not select features. Setting coefficients exactly to zero is what lasso does.

Q4. True. At $\lambda = 0$ both are ordinary least squares; as $\lambda \to \infty$ both go to $\beta = 0$. In between, ridge spreads weight across many small coefficients while lasso concentrates it on a few and zeroes the rest.

Q5. With 200 features and 100 observations, least squares can satisfy all 100 equations exactly, so it fits the training data perfectly, but the solution is one of infinitely many and carries no information about which five features matter. Ridge shrinks all 200 coefficients uniformly and still cannot tell the five from the 195, because the squared penalty prefers many small coefficients to a few large ones. Lasso's absolute-value penalty has no such preference, and its solutions set most coefficients exactly to zero; with λ chosen by cross-validation it recovers which five features matter.

Q6. True. Collinear columns make the least-squares coefficients unstable: many nearly-equivalent ways to split weight between the redundant features, and small changes in the data swing the split. The ridge penalty anchors the solution and stabilizes it. This is the collinearity example in the ridge notebook.

9.2 Where we are

Trees on CIFAR-10, accuracy against depth: shallow trees have a small train–test gap but low training accuracy (underfitting, not overfitting); deep trees have high training accuracy but a huge gap (overfitting, not underfitting). No depth is good. Before abandoning trees, we ask whether we can keep what is good about each end and repair its weakness. The repairs are completely different, because the problems are. Today: deep trees and **bagging**. I will describe it in terms of trees, but nothing about it is specific to trees.

9.3 Overfitting is high variance

Recall the picture from regularization. Nature is an IID data generator: this class is one random sample of students, next year's class is another. When I run `model.fit(X, y)`, the algorithm may be deterministic, but its input is random, so its output is random. For linear regression we drew the histogram of fitted slopes across many batches and called its width the variance.

Here is the bridge to today. Seeing very different *performance* on two batches of data (training and test) is what we have been calling overfitting. Getting very different *solutions* on two statistically identical batches is what we call high variance. These are the same phenomenon. A model that says $\beta = -5$ on one batch and $\beta = 20$ on the next is a high-variance model, and it is an overfitting model. So the question "how do I reduce overfitting?" becomes "how do I reduce variance?", and that is a question basic probability can answer.

9.4 Averaging reduces variance

Take a random variable X uniform on $\{1, 2, 3, 4, 5\}$. Its mean is 3. Its variance is the expected squared distance from the mean:

$$\mathrm{Var}(X) = \tfrac{1}{5}(1-3)^2 + \tfrac{1}{5}(2-3)^2 + \tfrac{1}{5}(3-3)^2 + \tfrac{1}{5}(4-3)^2 + \tfrac{1}{5}(5-3)^2 = \tfrac{1}{5}(4+1+0+1+4) = 2.$$

Now draw two independent copies X_1, X_2 and average them: $Z = \frac{1}{2}(X_1 + X_2)$. The mean of Z is still 3; averaging does not move the mean. But for Z to be far from 3, both X_1 and X_2 have to deviate in the same direction, which is less likely than one of them deviating. So the variance should drop. The calculation:

$$\mathrm{Var}(Z) = \mathbb{E}\Big[\Big(\tfrac{X_1+X_2}{2} - 3\Big)^2\Big] = \mathbb{E}\Big[\Big(\tfrac{X_1-3}{2} + \tfrac{X_2-3}{2}\Big)^2\Big] = \tfrac{1}{4}\,\mathbb{E}\big[(X_1-3)^2 + (X_2-3)^2 + 2(X_1-3)(X_2-3)\big].$$

(Pulling a constant a out of a variance gives a^2; that is where the $\frac{1}{4}$ comes from.) The cross term is the expectation of a product of independent quantities, so it is the product of the expectations, and each factor has expectation zero. It vanishes. What remains is $\frac{1}{4}(\mathrm{Var}(X) + \mathrm{Var}(X)) = \frac{1}{2}\mathrm{Var}(X)$. Averaging two independent copies halves the variance.

The same argument with n copies gives a factor $1/n^2$ outside, n variance terms inside, and all the cross terms dead by independence:

$$\mathrm{Var}\left(\frac{1}{n}\sum_{i=1}^{n} X_i\right) = \frac{1}{n}\mathrm{Var}(X).$$

Average 100 independent copies and the variance drops by a factor of 100. This is one form of the law of large numbers: flip a coin 1,000 times and the fraction of heads is very close to $\frac{1}{2}$, while ten flips can easily give you 20% or 30%.

Standard deviation is the square root of variance, so it drops by $1/\sqrt{n}$. To halve it, quadruple n. This is the same rule of thumb from last time, now derived in three lines. It matters because more data is one of the things you can pay for, and you want to know in advance what it buys.

A puzzle to think about: the calculation used independence to kill the cross terms. What happens if the X_i are identically distributed but *not* independent? Does the variance go down as fast, slower, or not at all?

9.5 Wishful thinking: average the trees

We now have two facts. Deep trees have low bias (they can fit the trend) and high variance (they overfit). Averaging independent random variables reduces variance. Let's connect them.

What is the random variable when we train a tree? The tree itself: the output of `fit`. Suppose, wishfully, that we had M independent datasets, M separate classrooms. Train a deep tree on each: $T_1, \ldots, T_M$. Would T_1 and T_2 look alike? No; that is what high variance means. Two draws from a uniform distribution on $[1, 10^6]$ are far apart; two draws from a Gaussian with mean 5 and standard deviation 0.005 are within about 0.01 of each other. Deep trees are the first kind. Their splits differ, their leaves differ, and no single one predicts next year's classroom well.

So average them. What does it mean to average trees? Not anything internal: there is no sense in averaging split thresholds or tree shapes. Average the *outputs*. For a new point x_{new},

$$\hat{y}(x_{\text{new}}) = \frac{1}{M}\sum_{m=1}^{M} T_m\texttt{.predict}(x_{\text{new}}).$$

For regression this is obvious: one tree says \$100,000, another says \$110,000, the average is \$105,000. For classification with 100 trees, if 75 say blue and 25 say red the average is "75% blue." That is a soft score, exactly what logistic regression gave us through the sigmoid, arrived at by a completely different mechanism. The hard label is blue, and the model is fairly confident about it.

9.6 The bootstrap

The problem with the wishful version is that we do not have M independent datasets. If we did, we would have M times the data and would not need any of this. The **bootstrap** manufactures them.

Given a dataset of 100 patients, make a new dataset of 100 patients by *sampling with replacement*: pick a patient at random, copy that row into the new dataset, put the patient back, pick again. Some patients appear twice or three times; some do not appear at all. Repeat the whole procedure M times to get M *bootstrap copies*.

Why with replacement? Sampling 100 rows from 100 rows *without* replacement gives you the original dataset in a different order, and none of our algorithms care about row order, so you would train the same tree M times. Sampling with replacement gives a dataset that is different from the original, but has the same statistics, in the same sense that two cohorts of students drawn from the same population are different individuals with the same demographics. The slides show ten bootstrap samples of $\{1, \ldots, 10\}$. The first is $[10, 1, 1, 3, 7, 5, 9, 9, 6, 9]$: 9 appears three times, 1 twice, and 2, 4, and 8 not at all. Same population, different sample.

Bootstrapping has a second use that we will not pursue today but that you should know about: if you want to know the *variance* of an algorithm — how much its output would change on a fresh batch of data — run it on many bootstrap copies and look at the spread of the results. It is the histogram experiment from the regularization lectures, done without access to nature.

The algorithm, now fully specified:

1. From the one dataset you have, draw M bootstrap copies $(X^{(m)}, y^{(m)})$.
2. Train a deep tree T_m on each copy.
3. Predict on a new point by averaging the M predictions.

This is **bagging** (bootstrap aggregating). Whether or not you yet believe it should work, it is a complete algorithm and it is a few lines of Python.

9.7 Why it works

Return to the synthetic square: blue to the right of $x_1 = 5$, red to the left, plus a few randomly flipped points. The trend is present in training and test; the flipped points are not. A depth-1 tree draws the vertical line. A deep tree draws the line and also carves out a box around each stray red point on the blue side, because it can, and that is overfitting.

Now take bootstrap copies. In some copies, one of the many blue points on the right is missing; that changes nothing, because the trend is carried by hundreds of points. In some copies, the one stray red point is missing, and in those copies the deep tree does not carve out its box. So across the M trees, every tree draws the vertical line, and only some of them draw the box. Average, and the boxed region comes out mostly blue.

That is the whole mechanism. Overfitting means "forget the trend, do whatever perverse thing gets zero training error." Each tree fits the trend the same way, because the trend is in every copy. Each tree overfits differently, because the outliers differ from copy to copy. Averaging keeps what they agree on and cancels what they don't. The numbers from the notebook make the case. On this data a depth-6 tree has training accuracy 0.96 and test accuracy 0.75, with training cross-entropy 0.078 and test cross-entropy 5.85: a large gap, both in accuracy and, more dramatically, in loss, since the deep tree is confidently wrong on the points it memorized. The shallow tree has training and test accuracy both 0.84 and cross-entropy 0.44 on both: no gap, and here, because the true trend really is one vertical line, no underfitting either, so it does better than the deep tree. Now take 20 bootstrap copies, train a depth-6 tree on each, and average: test accuracy 0.83, with decision

regions that look much more like a vertical line. We recovered nearly everything the deep tree threw away by overfitting, without telling it the answer was a line.

The same thing on CIFAR-10: the best single tree is around 24%; averaging 20 depth-6 trees trained on bootstrap copies gives about 34%. Not a good number for CIFAR-10, but the point is that we moved from a place where we were overfitting to a place where we overfit less, on a real problem.

Two remarks. First, bagging is a randomized algorithm: run it without a seed and you get a different answer, because the bootstrap copies differ. Second, the condition for averaging to help is that the models overfit *differently*. If every tree labels the boxed region red, the average labels it red. What made the trees differ here was only the presence or absence of one point in the data.

9.8 Bagging is not about trees

Nothing above used the fact that the model was a tree. You have some algorithm (a tree classifier, a logistic regression, a neural network) with a `fit` routine that somebody has already written, and it overfits. Bagging costs you one extra step: go from (X, y) to bootstrap copies (`sklearn` does this for you, though writing it yourself is not hard), then call `fit` on each copy and average the `predict` outputs. The expensive part, `fit`, you did not have to reinvent.

You can even average different algorithms: a tree, a logistic regression, and a network. If anything, that is better, because different model families overfit in different ways, and the more differently the components overfit, the more the averaging cancels. Analyst time is a bottleneck alongside data and computation, and bagging is cheap in all three senses.

9.9 Random forests

Bootstrap copies make the trees differ through the data. **Random forests** push the trees further apart by restricting what each one is allowed to do. The usual tree algorithm, at each node, picks the best feature to split on among all features. A random forest instead draws a random subset of the features at each node and picks the best split among only those. Different trees get different subsets. If your tree may only split on features 1 through 5 and mine only on 6 through 10, our trees will differ even on identical data.

This would be a catastrophe for a shallow tree: there may be one good split, and you have forbidden it. For a deep tree it is harmless: the tree will eventually get where it is going, just by a different route, and that different route is exactly what we want.

Bagging plus random feature subsets is the random forest. In `sklearn` it is

```
model = RandomForestClassifier(max_depth=15, n_estimators=100)
```

followed by the usual `fit`, `predict`, and `predict_proba`. That one line draws 100 bootstrap copies (one per estimator), draws feature subsets at every node of every tree, trains, and averages. Try it on CIFAR-10. I have not been able to get any tree-based method, including boosting, above 50% there; see how close you can get.

9.10 Two questions to close

Why not bag *shallow* trees? Because bagging cancels overfitting, and shallow trees are not overfitting; they are missing the trend. Averaging 100 models that all miss the trend gives you a model that misses the trend. Every component has to be able to find the trend for averaging to help.

And a puzzle for next time: what happens to *training* accuracy as you increase the number of trees in a bagged ensemble? Up, down, or the same?

Next time: the other end of the depth axis. Shallow trees do not overfit, and **boosting** is the way to combine them so that they stop underfitting.

Lecture 10

Boosting, and the idea of gradient descent

Thursday, September 24, 2026

Questions of the Day

Q1. True or False: Averaging N independent predictions with the same variance reduces the variance by a factor of N, but reduces the standard deviation only by a factor of $\sqrt{N}$.

Q2. True or False: Sampling every row exactly once without replacement creates a useful bootstrap sample because it gives a statistically similar but meaningfully different training data set.

Q3. True or False: Bagging combines trees by averaging their predictions on a new observation rather than by averaging their split locations or tree structures.

Q4. True or False: Bagging is just as effective for shallow trees that underfit as it is for deep trees that fit the trend but have high variance.

Q5. Free answer: Deep trees trained on different bootstrap samples tend to capture the same underlying trend but overfit different observations. Explain why averaging their predictions can improve testing performance.

Q6. True or False: A random forest restricts each tree to random subsets of features partly to make the trees less correlated, even if this prevents some individual trees from choosing the globally best split.

10.1 Discussion of the questions

The head-to-head questions are back. All of them, including the ones we did not get to last week, are posted; they are a good way to check that you are on board with the material.

Q1. True. We did this computation last time, and it used exactly two rules: the expectation of a sum is the sum of the expectations, and the expectation of a product of *independent* quantities is the product of the expectations. Everything else was expanding a square. The variance of the average is $\mathrm{Var}(X)/N$, and standard deviation is its square root. Why keep repeating this? Because it answers a question you will face: if I collect more data, what do I get? Not a different answer, necessarily, but a more trustworthy one, and this rule says how much more. To double your precision you need four times the data.

Q2. False. Without replacement you get the same rows in a different order, and no algorithm we use cares about row order. Sampling with replacement is the right way to think about it: nature is an infinitely large bag of possible experiments, and every real dataset is a handful fished out of it. Bootstrapping does the same fishing, but from the much smaller bag that is our dataset.

Q3. True. Averaging happens at the output, on a new point, and never at the level of parameters or tree structure. For linear regression you could at least imagine averaging the β's; for trees that split on feature 2 in one case and feature 7 in another, it is not even clear what averaging would mean. (A student proposed a tree that flips a coin at each node between the two splits. You could build that, but it is not bagging, and I would not expect it to work.) A fair correction from class: bagging is not only for trees. Any two models with a `predict` routine can be averaged.

Q4. False. Bagging averages models that were each trained to do as well as possible on the training data, without any coordination. If each one is missing the trend, averaging them does not find it. Follow-up: what happens to *training* error when you bag? It typically goes *up*. Each individual tree did its best on the training data, and there is no reason the average of several of them should do better on that same data. This is fine when you are bagging deep trees, whose training error is nearly zero and whose problem is the gap. We bag when we are overfitting. When we are underfitting we need a different tool, which is today's topic.

Q5. If every tree fits the trend, averaging preserves the trend. Each tree also carries artifacts from overfitting its own bootstrap copy, and because the copies differ, the artifacts differ, and on average they cancel. That is the whole argument, and the synthetic-square experiment is the illustration.

Q6. True. At every node, rather than searching all features for the best split, a random forest draws a random subset of size k and searches only those. This makes the trees less correlated, which is what averaging needs. It also costs something: as k shrinks, each tree's training error rises, and at $k = 1$ you are assigning split features at random with no reference to the data. The default is $k \approx \sqrt{\text{number of features}}$, about 55 for CIFAR-10's 3,072 features. Larger k: individual trees fit better but look more alike, so averaging removes less variance. Smaller k: worse individual trees, more different from each other, more variance removed. Another hyperparameter, another trade-off.

10.2 A table of knobs

It is worth pausing on where we now are. After the first week the only knob we had was tree depth: deeper means more overfitting and less underfitting, shallower means the reverse, and on CIFAR-10 no depth was acceptable. Now the table has many rows.

Turning up...	**overfitting**	**underfitting**
tree depth (in a forest)	↑	↓
number of bagged trees	↓	↑ (slightly)
feature-subset size k	↑	↓
number of engineered features (regression)	↑	↓
ridge penalty λ	↓	↑
lasso penalty λ	↓	↑ (differently)
number of boosted models (today)	↑	↓

The arrows point in different directions, and that is the point. We no longer have a one-dimensional search; we have several knobs with different effects, and combinations of them, with which to look for a setting where test performance is good. But using any of them requires the diagnosis first: are we overfitting, underfitting, or both? Everything keeps coming back to that.

10.3 Boosting: the other end of the depth axis

Bagging took deep trees, which do not underfit, and reduced their overfitting by averaging. Shallow trees have the opposite profile: they do not overfit, but they are too simple. Bagging cannot help them; they already have low variance. We need to combine simple models in a way that makes the combination *less* simple.

The idea. In bagging, every tree is trained independently, in parallel, and they meet only at the end to be averaged. In **boosting**, the second model is trained to *improve* the first, the third to improve the sum of the first two, and so on. The models are still added, but each one is built knowing what the previous ones got wrong. Nothing is specific to trees: boosting is for any simple model, where "simple" means "does not overfit." Boosting models that already overfit is a bad idea, for reasons that will become clear.

Boosting for additive loss (regression)

Call a loss *additive* if it depends only on the difference $y - \hat{y}$: squared loss $(y - \hat{y})^2$ and absolute loss $|y - \hat{y}|$ both qualify. Take squared loss. Step 1 is ordinary:

```
model1 = SimpleModel()     model1.fit(X, y)
```

where `SimpleModel` might be a depth-1 regression tree. Model 1 is a poor model, but it does not overfit.

Step 2. Declare `model2 = SimpleModel()`, same family (it need not be). What should we fit it to? Not to (X, y): then `model1 + model2` has no reason to beat `model1`. The goal is for the *sum* to have lower training error than Model 1 alone. (Training error, note, and that is fine, because neither model overfits, so lower training error should mean lower test error.) Write the training error of the sum and rearrange:

$$\sum_i \big(y_i - [\,\text{model}_1(x_i) + \text{model}_2(x_i)\,]\big)^2 \;=\; \sum_i \big(\underbrace{[\,y_i - \text{model}_1(x_i)\,]}_{r_{1,i}} - \text{model}_2(x_i)\big)^2.$$

Nothing happened except distributing a minus sign, but look at what it reveals. The bracket $r_{1,i} = y_i - \text{model}_1(x_i)$ is the **residual**: the error Model 1 makes on point i. It is a fixed number, because y_i is data and Model 1 was already trained. So the expression is exactly what `model2.fit` minimizes when you hand it X and the residuals as the target. If Model 2 could predict the residual perfectly, the sum would equal y exactly and training error would be zero. So:

```
r1 = y - model1.predict(X)     model2.fit(X, r1)
```

In the housing example: Model 1 tries to predict the price. Model 2 takes the same houses and tries to predict not the price but the gap between Model 1's prediction and the truth. It is trying to fix Model 1.

Then iterate: `r2 = r1 - model2.predict(X)`, `model3.fit(X, r2)`, and so on for T steps. The final prediction is the sum of all T models. It is a for loop.

Why sequentially, and not all at once?

Suppose the goal is two depth-3 trees T_1, T_2 whose sum has minimal training error. The procedure above does not find the best pair. It finds the best T_1 as if it were alone, freezes it, and then finds

the best T_2 given T_1. Searching over both simultaneously would do better, or at least no worse. (One student's analogy: if you know a friend is coming to help, you would divide the task rather than attempt all of it yourself. Model 1 does not know Model 2 is coming.) So why don't we?

The mundane answer: there is no code for the simultaneous version. The sequential version is written above in four lines, using nothing but a `fit` routine somebody else already wrote. To find the best sum of two trees jointly, you would have to invent and implement a new procedure, and then a different one for three trees, and another for a tree plus a neural network. Implementation time is a bottleneck alongside data and computation. Boosting and bagging both respect it: they are built from LEGO blocks that already exist. Anything with a `fit` routine can be bagged, and anything with a `fit` routine for an additive loss can be boosted, including mixtures — a depth-4 tree, then a small network, then a linear regression.

The table, for boosting

As the number of boosted models grows, underfitting goes *down* (that is the goal) and overfitting goes *up*. These are the reverse of bagging's arrows, and the logic is symmetric. Bagging starts at the deep-tree extreme, where underfitting is negligible and overfitting is the problem, and is willing to give up a little training accuracy to shrink the gap. Boosting starts at the shallow-tree extreme, where overfitting is negligible and underfitting is the problem, and is willing to open a small gap in exchange for fitting the trend.

10.4 Boosting for classification: exponential loss

Everything above used one trick, $y - (m_1 + m_2) = (y - m_1) - m_2$, which requires the loss to be a function of the difference. No classification loss is additive. 0/1 loss is 0 if the labels match and 1 otherwise; nothing to subtract. So take one classification loss and see how to rescue the idea.

Exponential loss for binary classification. Label the classes $y \in \{-1, +1\}$ (the names are arbitrary, but this choice makes the algebra work). The model outputs a real number $z = \text{model}(x)$; the hard label is the sign of z, so the decision boundary is $z = 0$ as with cross-entropy. The loss on a point is

$$\ell(y, z) = e^{-yz}.$$

If $y = +1$ and z is large and positive, the loss is $e^{-\text{large}}$, tiny: confidently correct is rewarded. If $y = +1$ and z is large and negative, the loss is $e^{+\text{large}}$: confidently wrong is punished hard. Same character as cross-entropy, different formula. Note that the best model does not output ± 1; it outputs huge positive numbers on positive points and huge negative numbers on negative points.

Step 1 as before: `model1.fit(X, y)`. For step 2, write the training loss of the sum and use $e^{a+b} = e^a e^b$:

$$\sum_i e^{-y_i[\,\text{model}_1(x_i)+\text{model}_2(x_i)\,]} \;=\; \sum_i \underbrace{e^{-y_i\,\text{model}_1(x_i)}}_{w_{1,i}}\; e^{-y_i\,\text{model}_2(x_i)}.$$

The first factor, $w_{1,i}$, is a fixed number once Model 1 is trained; call it a **weight**. When is it big? When $\text{model}_1(x_i)$ has the wrong sign, that is, when Model 1 got point i wrong; and it is tiny when Model 1 was confidently right. If all the weights were 1, minimizing this sum would be exactly `model2.fit(X, y)` under exponential loss. With the weights, it is a *weighted* fit: minimize $\sum_i w_i\, \ell(y_i, \text{model}_2(x_i))$.

Read what that says. Model 2 is told to fit (X, y), the original labels, not a residual, but with its attention redirected: points Model 1 got confidently right have weight near zero, as if deleted; points Model 1 got wrong dominate the loss. It is an exam where some questions are worth 100 points and others 0.1; you spend your effort on the 100-point questions and do not care if you miss the others. The weights are set not by the exam but by how the previous model did.

So for classification we need a slightly richer building block: a `fit` that accepts a weight vector, `model.fit(X, y, w)`. Given that, the loop is

```
model1.fit(X, y, ones)
w1 = exp(-y * model1.predict(X))
model2.fit(X, y, w1)
w2 = w1 * exp(-y * model2.predict(X))
model3.fit(X, y, w2)    ...
```

The first weights are all ones because Model 1 has no information about where the difficulties are. After that, each weight vector is the previous one multiplied by the new model's per-point exponential loss; points that keep being missed keep gaining weight. Again a for loop, again nothing beyond existing routines and a few lines of NumPy.

10.5 Toward gradient boosting

You should not be satisfied yet. We have said all semester that cross-entropy is the loss for classification, and I have just shown you boosting for exponential loss. Are we going to do a separate derivation, with its own trick, for every loss? That is a fair complaint. **Gradient boosting** answers it: do the derivation once, for a general loss, requiring only that the loss be differentiable. (That is the G and the B in XGBoost, one of the most widely used implementations, and the model I predict most of you will end up using on Kaggle.)

Set up the general step. Model 1 predicts $\hat{y}$; we want Model 2 to predict some change δ so that the loss of $\hat{y} + \delta$ is lower than the loss of $\hat{y}$. For squared loss the right δ was the residual; for exponential loss it was hidden in the reweighting. For a general loss we can play neither trick, so we need a general tool for "move so as to decrease a function." That tool is gradient descent, and since it is also how we will train neural networks, it is worth an interlude.

10.6 Interlude: gradient descent

The problem: a function of one variable x (think of it as a loss; lower is better), and you want its minimum. You can see the whole curve and point to the bottom. An algorithm cannot. It knows the function only *locally*, at the point where it stands.

The intuition is to roll downhill. A ball placed on a surface knows nothing about the global shape of the surface; it knows which way gravity pulls it right now, and following that local information gets it to the bottom. Standing on a mountain in fog, you cannot see the valley, but you can feel which direction descends fastest and step that way. We use gradient descent for everything at scale because, with ten million parameters, local information is all we can afford to compute. And recall from Lecture 2 why we insisted on smooth losses: cross-entropy, exponential, and hinge losses are shaped so that rolling downhill reaches the minimum; 0/1 loss has ridges you would have to climb over, so rolling downhill gets stuck.

To make "roll downhill" algorithmic: at the current point, compute the tangent line. Its slope is the **derivative**; in more than one variable it is called the **gradient**, written ∇f, and it is just the vector of slopes in each coordinate direction. If the slope is negative, the function decreases to the right, so move right; if positive, move left. In one line,

$$x_{\text{new}} \;=\; x_{\text{old}} \;-\; \eta\, \nabla f(x_{\text{old}}),$$

where η is a small positive number, the **step size**. Follow the negative gradient, and take a small step. Why small? Look at the curve: a large step from a point on the left could overshoot the valley and land higher on the right. On the mountain, a hundred-mile step might put you on top of a taller mountain. In two variables the picture is a topographic map, and gradient descent walks perpendicular to the contour lines.

We will need more than one variable. Linear regression has β_0 and β_1; the housing model had four parameters; a neural network has millions. The gradient handles all of them at once, and Python computes it; we do not have to. Someone asked about functions with many local minima, where rolling downhill can end in the wrong valley. Real. The examples today have a single valley; the harder cases are for later.

Next time: how gradient descent gives us boosting for any loss, then a summary of bagging and boosting, and a third ensembling method, stacking.

Lecture 11

Gradient boosting

Tuesday, September 29, 2026

Questions of the Day

Q1. True or False: Boosting is most useful when each base learner already fits the training data closely and overfits individual points, because later models can directly correct those overfitting mistakes.

Q2. True or False: In regression boosting with squared loss, after Model 1 is fit, Model 2 should be trained to predict the residuals Y minus Model 1's predictions rather than the original targets Y.

Q3. True or False: Adding more boosted estimators generally reduces underfitting while increasing the risk of overfitting, whereas averaging more bagged deep trees can worsen training fit while reducing overfitting.

Q4. True or False: Under exponential loss with labels equal to -1 or $+1$, a large positive score on a point labeled $+1$ receives less loss than a small positive score on that same point.

Q5. Free answer: A depth-1 regression tree underfits the training data. Explain how boosting with squared loss adds a second tree: what should the second tree predict, and why can the sum of the two trees have lower training error?

Q6. True or False: In exponential-loss classification boosting, observations that Model 1 classified confidently and correctly receive larger weights when Model 2 is trained than observations that Model 1 classified incorrectly.

11.1 Discussion of the questions

Today's plan: finish boosting by making it work for any loss, using gradient descent. Gradient descent will come back later in the semester when we train neural networks and no longer have the luxury of a `model.fit`. Neural networks start Thursday or next week and stay with us to the end.

A word on vocabulary first. The individual models that get boosted (or bagged) are called **base learners**: if you boost trees, the trees are the base learners.

Q1. False, and the wording has to be untangled. "Already fits the training data closely" means low training error, which means not underfitting. Boosting is for underfitting; it can only improve the fit to the training data, and if the fit is already good there is nothing for it to do. "Overfits individual points" is the picture from the synthetic square: deep trees carving boxes around flipped labels. That is the situation for *bagging*, where a bootstrap copy that happens to omit the stray point produces a tree that ignores it, and averaging washes the boxes out.

This led to a longer discussion worth recording. Suppose a base learner both underfits and overfits

(high training error and a large train–test gap, which we know is possible, for instance a depth-6 tree on CIFAR-10). Is it a good candidate for boosting? No. Boosting fixes underfitting and *causes* overfitting. If you start with a learner that already overfits, boosting makes that worse. So we boost learners that overfit not at all, or barely: very shallow trees. Symmetrically, bagging fixes overfitting and cannot fix underfitting; averaging models that miss the trend produces a model that misses the trend, and training error typically goes up. So we bag learners that do not underfit: deep trees. Starting from a learner with the wrong deficiency puts a ceiling on what the ensemble can reach. Try it anyway, in the homework, to see the effect for yourself.

One more subtlety. Overfitting is easy to measure: it is the gap between training and test error. Underfitting is harder, because "training error too high" only makes sense relative to a baseline. On CIFAR-10 we have one: we can classify these images ourselves, so we know 95% or better is achievable and 60% training accuracy means the model is missing signal that is there. On an unfamiliar problem you need some notion of what is achievable before you can say you are underfitting. That makes underfitting the more delicate diagnosis.

Q2. True. Model 1 is trained on (X, y). Model 2 is trained on (X, r_1) where $r_1 = y - \text{model}_1(X)$. We did not assume this; we derived it. Wanting $y - (m_1 + m_2)$ small is the same as wanting $(y - m_1) - m_2$ small, which says m_2 should fit the residual.

Q3. True. This is the summary of the past two lectures. More boosted estimators: less underfitting (the reason we boost), more overfitting. More bagged deep trees: training fit may worsen (we are averaging models that were each trained separately), overfitting shrinks (the reason we bag). You will tune the number of estimators on the next problem set; it is one of the hyperparameters of the boosting routine you will likely use on Kaggle.

Q4. True, and it is worth working through what exponential loss actually does, because the answer is less obvious than it looks.

A worked example

Three data points with two features and labels in $\{-1, +1\}$:

x_1	x_2	y
1.0	−0.6	−1
1.4	0.3	−1
0.8	0.1	+1

A depth-1 tree needs four things: which feature, what threshold, and the two leaf values L_1 (if the test is false) and L_2 (if true). The class chose to split on x_1 at 0.9 (anything between 0.8 and 1.0 works). Point 3 goes to leaf 1; points 1 and 2 go to leaf 2. The partition is perfectly homogeneous, so the split is as good as it gets.

For the leaf values, the first suggestion was $L_1 = 0.33$, $L_2 = 0.66$. Those would be sensible under some losses. Under exponential loss, $\ell(y, h(x)) = e^{-y\,h(x)}$, the total loss is a sum of three terms, one per row:

$$e^{-(-1)(0.66)} + e^{-(-1)(0.66)} + e^{-(+1)(0.33)} \;=\; e^{0.66} + e^{0.66} + e^{-0.33}.$$

Two of the three terms are e^{positive}: the leaf values have the *wrong sign.* Exponential loss wants $y \cdot h(x)$ positive, so that the loss is $e^{-\text{positive}}$ and small; the hard label is the sign of $h(x)$, and here it would call points 1 and 2 positive. So first fix the signs: $L_1 > 0$, $L_2 < 0$. Then ask how large. The loss is $e^{-L_1} + 2e^{L_2}$, and it keeps decreasing as $L_1 \to +\infty$ and $L_2 \to -\infty$. There is no minimizer.

This is the same phenomenon we saw with logistic regression on separable data: when every point can be classified correctly, a loss that rewards confidence drives the scores to infinity. (That is Q4: a larger positive score on a $+1$ point means smaller loss, with no limit.)

Now make it harder. Add two points, $(0.4, 0.6, +1)$ and $(0.2, 0.9, -1)$, and insist on the split $x_1 \geq 0.7$. Points 4 and 5 fall in leaf 1, one positive and one negative. The loss on leaf 1 is $e^{-L_1} + e^{L_1}$. Intuition says $L_1 = 0$: you cannot agree in sign with both, so say nothing. Calculus agrees: the derivative is $-e^{-L_1} + e^{L_1}$, zero when $e^{L_1} = e^{-L_1}$, i.e. $L_1 = 0$.

Leaf 2 holds points 1, 2, 3: two negatives and one positive. We expect L_2 negative, since the majority is negative, but not $-\infty$, since that would cost a huge loss on point 3. The loss on leaf 2 is $2e^{L_2} + e^{-L_2}$. Derivative $2e^{L_2} - e^{-L_2} = 0$ gives $2e^{L_2} = e^{-L_2}$, so $\ln 2 + L_2 = -L_2$ and

$$L_2 = -\tfrac{1}{2} \ln 2 \approx -0.35.$$

Negative, as expected, and modest. With three negatives and one positive it would be more negative: exponential loss will accept being confidently wrong on one point in exchange for more confidence on three. Compare 0/1 loss on the same leaf: it would simply pick the majority, $L_2 = -1$, and would be indifferent to magnitude. Different loss, different leaf value, different character of solution. That is the same lesson as Lecture 2, now with numbers.

Q5. The first tree is trained to map X to y and does so poorly, because it is a stump. Compute the residual $r_1 = y - \text{model}_1(X)$: not where the first tree was wrong, but by how much, at each point. Train the second tree to map X to r_1. The sum of the two trees then predicts $\text{model}_1(x) + \text{model}_2(x) \approx \text{model}_1(x) + r_1 = y$; to the extent the second tree captures the residual, the training error of the sum is lower than the training error of the first tree alone.

Q6. False; it is the reverse. Picture seven points, three positive and four negative, and suppose Model 1 is a line that gets the three positives confidently right, three of the negatives confidently right, and one negative wrong. Model 2 is trained on the same (X, y) but with a weighted loss, $\sum_i w_i \, \ell(y_i, \text{model}_2(x_i))$, and the weights come from Model 1's per-point loss. The three confident positives get weights near zero, as though erased. The three correct negatives get small weights. The misclassified negative gets a large weight; it would be larger still had Model 1 been confidently wrong on it. A weight of zero deletes a point; a weight a thousand times larger than the others tells `fit` that this is the exam question worth ten points when the others are worth 0.01. The weights do not say "ignore everything else"; they say "fix this one first, and do not break the others to do it." That is the iterative refinement.

The derivation, once more, since it is the heart of the matter. The training loss of the sum is $\sum_i e^{-y_i(m_1(x_i)+m_2(x_i))} = \sum_i e^{-y_i m_1(x_i)} \cdot e^{-y_i m_2(x_i)}$. Model 1 is frozen, so the first factor on each point is a known number, w_i. Minimizing over Model 2 is then a weighted exponential-loss fit, `model2.fit(X, y, w1)`. Ordinary `fit` is the special case where every weight is one: ten exam questions, all worth the same.

11.2 Gradient boosting: boosting for any loss

We now have boosting for additive losses (fit the residual) and for exponential loss (reweight the points). Each used a trick specific to the loss. We do not want to repeat this for hinge loss, for cross-entropy, and for every loss anyone invents. So let us ask the question abstractly.

Model 1 outputs predictions $\hat{y} = (\hat{y}_1, \ldots, \hat{y}_n)$ on the training points, with total loss $\sum_i L(y_i, \hat{y}_i)$. Model 2 will add something: the new predictions are $\hat{y} + \Delta$ for some vector $\Delta = (\delta_1, \ldots, \delta_n)$. What do we want Δ to be? Whatever makes $\sum_i L(y_i, \hat{y}_i + \delta_i)$ smaller than $\sum_i L(y_i, \hat{y}_i)$. For squared loss we could solve this exactly (δ_i should be the residual). In general we cannot. But "I am at a point, how should I perturb it to decrease a function?" is exactly the question gradient descent answers.

Gradient descent, once more, with numbers

Take $f(x) = -2x^2 + 5x + 1$ and suppose you are standing at $x = 1$, where $f(1) = 4$. Should you move to 1.1 or to 0.9? You could evaluate both. The method that scales is to compute the slope: $f'(x) = -4x + 5$, so $f'(1) = 1 > 0$. The function is increasing where you stand, so to go downhill, step *left*, to 0.9. In symbols, $x_{\text{new}} = x - \eta f'(x)$ with a small η: the step is $-\eta \cdot 1$, negative.

Now make the function nastier: add $3x^4 - 2x^3$ to it. None of us wants to find the global minimum of a quartic by hand. But the question "from $x = 1$, which way is downhill?" is still answered by one derivative evaluated at one point, and the step is still $-\eta$ times that number. You never need to understand the whole function. You need the slope where you are, and a small step back against it. With ten million parameters and a loss surface nobody can picture, that is all we will ever have, and it is enough.

A good question from class: how do you choose η? There is no clean general answer. Too large and you overshoot (on a parabola you can land higher than you started; on a mountain, a ten-mile step can put you on the next peak). Too small and progress is glacial. And after each step you recompute the slope, because it may now point somewhere else.

The algorithm

Return to boosting. Gradient descent says the ideal update to the predictions is

$$\hat{y} + \Delta \;=\; \hat{y} - \eta \, \nabla_{\hat{y}} \sum_i L(y_i, \hat{y}_i),$$

that is, each δ_i should be $-\eta$ times the slope of the total loss with respect to the i-th prediction. Call that vector of slopes the **gradient** $G_1 = \nabla_{\hat{y}} \sum_i L(y_i, \hat{y}_i)$. This is what we *wish* Model 2 would output: $-\eta \, G_1$.

What we *can* have is any function from the base-learner family trained by `fit`. So train it to output the gradient:

```
model2.fit(X, G1)
```

(absorbing the sign and η outside), and update the predictions to $\hat{y}^{(2)} = \hat{y}^{(1)} - \eta \, \text{model}_2(X)$. Then compute the gradient G_2 of the loss at $\hat{y}^{(2)}$, train `model3.fit(X, G2)`, update to $\hat{y}^{(3)} = \hat{y}^{(2)} - \eta \, \text{model}_3(X)$, and so on for T steps. The output is

$$\text{model}_1 \;-\; \eta \, \text{model}_2 \;-\; \eta \, \text{model}_3 \;-\; \cdots \;-\; \eta \, \text{model}_T.$$

This is **gradient boosting**. Where before we fed `fit` the residual, or the reweighted data, we now feed it the gradient. The workhorse is still `model.fit`, called over and over; the only new ingredient is a routine that computes the gradient of the loss, which Python will do for us. As long as that gradient exists, the same for loop works for squared loss, exponential loss, hinge loss, cross-entropy, or anything else. That is what the G in XGBoost means.

A check: for squared loss, $L(y, \hat{y}) = (y - \hat{y})^2$ has slope $-2(y - \hat{y})$ in $\hat{y}$, so the negative gradient is proportional to the residual. Gradient boosting with squared loss *is* residual fitting. The general method contains the special case we started from.

11.3 Bagging and boosting, side by side

The slides end with a table that is worth knowing cold. Two hyperparameters, depth of the base trees and number of trees, and three quantities: training error, test error, and the gap.

		Training error	**Test error**	**Gap**
Bagging	increase depth	down	cannot say	up
	more trees	up	cannot say	down
Boosting	increase depth	down	cannot say	up
	more trees	down	cannot say	up

Depth behaves the same way in both: deeper base learners fit the training data better and widen the gap. The number of trees is where they differ. Bagging more trees averages more independently trained models, which tends to push training error *up* and the gap *down*; that is why we bag, and why we bag deep trees whose training error we can afford to give up. Boosting more trees drives training error *down* and the gap *up*; that is why we boost, and why we boost shallow trees whose gap we can afford to open. Rule of thumb: bag to reduce overfitting, boost to reduce underfitting.

Now the column we have not discussed, and it is the only one we actually care about: *test error*, and every entry says "cannot say." We can predict what each knob does to training error and to the gap, but not to their combination. So how are you supposed to decide between 100 trees of depth 3 and 150 trees of depth 4? The answer is the one it has been since Lecture 4. Hyperparameters are chosen by a train/test split, or better, by cross-validation; more data-efficient, more computationally expensive. The table tells you which direction to turn a knob once you have diagnosed overfitting or underfitting. Cross-validation tells you where to stop.

We did not reach the last slide, on stacking; Thursday.

11.4 On the midterm

Asked about the format: short answer and some multiple choice, on paper, closed notes, and it will look a lot like the morning questions, including true/false with an explanation. The questions and this write-up are all posted. If you are comfortable with the daily questions and variations on them, you are prepared.

Lecture 12

Gradient boosting in practice, hyperparameters, and stacking

Thursday, October 1, 2026

Questions of the Day

Q1. **True or False:** In gradient boosting, after Model 1 is fixed, Model 2 should always be trained on the original labels Y because every new model must predict the true outcome directly.

Q2. **True or False:** Under exponential loss with labels equal to -1 or $+1$, if a tree leaf contains exactly one observation of each label, the constant leaf score that minimizes their combined loss is 0.

Q3. **True or False:** For the same fixed tree partition, minimizing 0–1 loss and minimizing exponential loss can produce different optimal constant scores for a leaf containing both positive and negative labels.

Q4. **True or False:** If every observation in a tree leaf has label $+1$, exponential loss has a finite minimum when the leaf score equals $+1$.

Q5. **True or False:** Averaging more bagged deep trees generally lowers training error on the full original training set for the same reason that adding more boosted trees lowers training error.

Q6. **Free answer:** Gradient boosting replaces loss-specific tricks such as residual fitting with a general procedure. What target should Model 2 fit at each stage, and why can the same `model.fit` pattern work with many different loss functions?

12.1 Discussion of the questions

Today we finish ensembling: a review of gradient boosting, then the practical question of choosing its hyperparameters, then **stacking**, the third and simplest of the three ensembling techniques. Two notebooks on bagging and boosting with XGBoost are posted; we will look at them Tuesday.

A word on packages. It is instructive to implement bagging and boosting yourself, and we have seen that it takes only a few lines, but in practice you will use a stable library. XGBoost is the one I use most; CatBoost is also very good; LightGBM is another. Until about a year ago, packages like these were by a wide margin the dominant approach to *tabular* problems, meaning data organized as rows and feature columns, like the diabetes dataset, without the structure of an image or a sentence. Tabular data had resisted neural networks. That has started to change with what are called tabular foundation models, which are now competitive with carefully tuned XGBoost; we will come back to them at the end of this semester or the start of next. Boosted trees remain very much worth knowing.

Q1. False. The phrase "gradient boosting" is there to distract. Boosting of any kind is iterative refinement, and the later models are not trained to predict Y directly. For additive loss, Model $k+1$ is trained on the residual, Y minus the sum of the first k models' predictions. For gradient boosting it is trained on the gradient of the loss with respect to the current predictions. The one case where you could argue for "true" is exponential-loss boosting, where Model 2 does see the original labels, but with the points reweighted; that is still refinement, not direct prediction. (Bagging is the setting where every model is trained on the original labels.)

Q2. True, and this is the easy version of last week's calculation. First, what the question is asking. A tree partitions the plane into rectangles, as many as 2^d for depth d, and every point in a rectangle gets the same output: the value L stored at the corresponding leaf. The question is what L should be when the rectangle contains one $+1$ and one -1. The loss on that leaf is $e^{-L} + e^{L}$, symmetric in L, minimized at $L = 0$. You cannot agree in sign with both points, so say nothing.

Now the trap that sits next to this question. Same leaf, but both points are $+1$. The loss is $e^{-L}+e^{-L}$, which decreases forever as $L \to +\infty$; the minimizer is $L = +\infty$. Why? Recall how exponential loss is used: the hard label is the sign of the model's output (the threshold is 0, where logistic regression's was 1/2), and the magnitude is the model's confidence. A leaf that sees only positive examples, asked what it thinks, says "positive, and I am infinitely sure," unless regularization forbids large values. We have seen this before, with logistic regression on separable data. Exponential loss rewards confidence, and when nothing in the data contradicts the model, confidence has no ceiling.

The intermediate case from last time, two $+1$'s and one -1: the loss is $2e^{-L} + e^{L}$. Driving $L \to +\infty$ makes the first two terms vanish and the third explode. Setting $L = 0$ gives loss 3. Somewhere to the right of 0 the loss dips below 3 before climbing, and that dip is at $L = \frac{1}{2}\ln 2 \approx 0.35$ (last time's example had the signs reversed, hence $-\frac{1}{2}\ln 2$). With three $+1$'s and one -1, the answer moves further right. The loss behaves the way intuition says it should: more agreement, more confidence. What takes computation is the exact rate at which confidence grows, and that depends on how the particular loss penalizes being confidently wrong against being correct but timid. Exponential loss trades those off one way; cross-entropy trades them off another way; both are trying to do the same intuitive thing. The practical point is that you cannot say what a model's output *is* without knowing which loss it was trained under.

Q3. True. This is the same point. Two $+1$'s and one -1 in a leaf: 0–1 loss picks the majority, $L = +1$, and is indifferent to magnitude; exponential loss gives $\frac{1}{2}\ln 2$. Change the loss, change the answer. We have now seen this in many forms.

Q4. False. As just discussed, with all labels $+1$ the loss has no finite minimizer; it decreases toward zero as $L \to +\infty$. (The question was not written as clearly as it should have been; the intended claim is that the minimum is finite and attained at $L = +1$, and that is what is false.)

Q5. False. Adding a boosted tree lowers training error because that is the new tree's entire purpose: it is fit to the residual, or the gradient, of the current ensemble. Adding a bagged tree has a different purpose. The new tree is fit independently on its own bootstrap copy and knows nothing about the others; averaging it in is meant to reduce variance, in the hope of doing better on test data. It typically does *not* lower training error, and may raise it.

Q6. Covered in the next section, since it deserves the board.

12.2 Gradient boosting, worked once more

Every boosting procedure begins the same way: `model1.fit(X, y)`, giving predictions $\hat{y}^{(1)}$ from `model1.predict(X)`. The question is always what goes in the second slot of `model2.fit(X, ?)`. For additive loss we put the residual. In general, write the predictions after two stages as

$$\hat{y}^{(2)} = \hat{y}^{(1)} + \text{model}_2(X),$$

which is just the definition of boosting, and ask: if I could add *any* perturbation Δ to $\hat{y}^{(1)}$, what would I want it to be?

Two pieces of bookkeeping make this cleaner. First, the training loss is a sum over data points, $\sum_i L(y_i, \hat{y}_i)$, but for this purpose think of it as one function of the whole prediction vector, $\text{Loss}(\hat{y})$. Second, the true labels are fixed; the predictions are the only thing we may change. So we are standing at a point $\hat{y}^{(1)}$ on a loss surface and want to step to $\hat{y}^{(1)} + \Delta$ with lower loss. That is gradient descent's question, and its answer is: step a small amount against the slope,

$$\Delta \;=\; -\eta \, \nabla_{\hat{y}} \, \text{Loss}(\hat{y}^{(1)}).$$

This is the *aspirational* Δ. We cannot simply set Model 2's output to it, because Model 2 comes from a limited family and may not be able to reproduce those numbers, any more than Model 1 could reproduce y exactly. What we can do is ask `fit` to get as close as it can. Absorbing the $-\eta$ into how we combine the models, the recipe is:

1. `model1.fit(X, y)`; set $\hat{y}^{(1)}$.
2. Compute the gradient $G_1 = \nabla_{\hat{y}} \, \text{Loss}(\hat{y}^{(1)})$. This is the one thing we must supply; in practice Python computes it.
3. `model2.fit(X, G1)`; set $\hat{y}^{(2)} = \hat{y}^{(1)} - \eta \, \text{model}_2(X)$.
4. Compute G_2 at $\hat{y}^{(2)}$; `model3.fit(X, G2)`; and so on.

Where before `fit` was handed the residual or the reweighted data, it is now handed the gradient. Nothing else changes. That is the answer to Q6: the target at each stage is the gradient of the loss with respect to the current predictions, and the same pattern works for any loss because the only loss-specific ingredient is that gradient, which exists for any differentiable loss.

The gradient for squared loss, explicitly

Take three data points and squared loss, so

$$\text{Loss}(\hat{y}) = (y_1 - \hat{y}_1)^2 + (y_2 - \hat{y}_2)^2 + (y_3 - \hat{y}_3)^2.$$

The gradient is the vector of partial derivatives with respect to $\hat{y}_1, \hat{y}_2, \hat{y}_3$, treating the y's as constants. By the chain rule the first component is $-2(y_1 - \hat{y}_1)$, the second is $-2(y_2 - \hat{y}_2)$, the third likewise. So

$$G_1 = -2\,(y - \hat{y}^{(1)}) = -2 \times \text{residual}.$$

Model 2 is fit to -2 times the residual, and the ensemble outputs $\text{model}_1 - \eta \, \text{model}_2 \approx \text{model}_1 + 2\eta \, \text{residual}$. With $\eta = \frac{1}{2}$ the constants cancel and we recover exactly the additive-loss procedure from the first day of boosting. Gradient boosting is the general method; residual fitting is what it does for squared loss.

12.3 Hyperparameters of boosting, and how to choose them

A third knob

So far boosting had two hyperparameters: the depth of the base trees and the number of estimators (sometimes called the number of learners). Gradient boosting adds a third, the **step size** η, also called the **learning rate**. Too small and a hundred trees may not make enough progress. Too large and each step overshoots. A tiny step size can also make overfitting slightly worse, by letting the procedure settle into small local valleys. You will set all three when you call XGBoost.

How should you choose them? One suggestion from class was a schedule, starting with a larger step and shrinking it. That is a real technique. But the general answer is the one we give every class: hyperparameters are chosen by cross-validation (or at least a train/test split); parameters are chosen by `fit`.

Why not always cross-validate?

If cross-validation gives a better answer than a single train/test split, why would anyone use the split? Computation. Compare a 95/5 split with 20-fold cross-validation: the latter costs twenty fits. If one fit takes half an hour, that is ten hours, and if you have to submit tomorrow, or even in five days, it limits how many things you can try.

To see how fast this adds up, take a modest grid for boosting: depths 2, 3, 4, 5; number of estimators 50, 100, 150, 200, 250, 300, 350, 400; learning rates 0.5, 0.1, 0.05, 0.01. That is $4 \times 8 \times 4 = 128$ combinations. With a single train/test split, 128 fits. With 20-fold cross-validation, about 2,500. Even at a few seconds per fit, including the time just to load data and print anything to the screen, you are into hours. And this grid ignores the L1 and L2 regularization parameters, which would multiply it again.

Three ways to cope

Fewer folds. Five folds instead of twenty is four times cheaper, and forty minutes versus ten is the difference between trying several things in a sitting and trying one. The cost is subtler, and worth understanding. Here is how hyperparameters are actually used, on Kaggle or anywhere else. You have $(X_{\text{train}}, y_{\text{train}})$ and X_{test}, and must submit $\hat{y}_{\text{test}}$. You use cross-validation to find, say, depth 4, 200 estimators, learning rate 0.05. Then you *retrain on 100% of your training data* with those settings, and predict on the test set. The splitting was only a device for choosing hyperparameters. Now notice: 3-fold cross-validation found the best hyperparameters for a model trained on two-thirds of the data; 100-fold found them for a model trained on 99%. The second resembles the final model much more closely. What you actually want is the best hyperparameters for training on 100%, which no procedure can give you; large K approximates it best. So bigger K is statistically better and computationally worse, and reducing folds is a real trade.

A smaller grid, coarse to fine. The multiplication is what hurts. Rather than eight values of the number of estimators, try 50, 250, 500; if 250 wins, come back with 200, 250, 300. Do not pad the grid with values you have no intention of using.

Let the library orchestrate. `GridSearchCV` in `sklearn` takes the grid and the number of folds and runs the whole thing. It does not make the computation cheaper, but it keeps you from writing nested loops by hand.

These are not abstract concerns. The numbers above show that you can create a computational bottleneck out of problems that each take two seconds to solve.

12.4 Ensembling is not about trees

Before the third technique, the point I keep repeating. Bagging: copy the data by sampling with replacement, train a model on each copy, average. Nothing says "tree." Boosting: fit a model, then refine it with another model fit to the gradient. The first model could be linear regression, the refinement a tree, or boosted trees, or a random forest, or a neural network. The only requirement is a `fit` routine for each piece, and next week we will write our own `fit` for neural networks, at which point nothing stops us from boosting those too. There is even a reason to want to: a random forest and a neural network make mistakes in very different ways, and combining models with different failure modes is exactly when ensembling pays.

In the language of the bias–variance trade-off: boosting reduces bias at the cost of variance; bagging reduces variance at the cost of bias.

12.5 Stacking

The third technique starts the same way as the others: `model1.fit(X, y)`, then compute the predictions $\hat{y}^{(1)}$ on the training data. Here Model 1 is *any* model; not "complex, low underfitting" as in bagging, not "simple, low overfitting" as in boosting.

Step 2: append the predictions to the data as a new column,

$$X^{(2)} = [\, X \mid \hat{y}^{(1)} \,],$$

and fit a second, possibly different, model to $(X^{(2)}, y)$. Compute its predictions $\hat{y}^{(2)}$ and append those too,

$$X^{(3)} = [\, X \mid \hat{y}^{(1)} \mid \hat{y}^{(2)} \,],$$

and repeat as desired.

Recognize what this is. In Lecture 7 we added a column $x_3 = x_1^2 + x_2^2$ so that logistic regression could separate a disk from a ring. Feature engineering, in general, is: take X, compute something from it, add the result as a new column. Once Model 1 is trained, `model1.predict` is exactly such a computation, from X to a new number. We did not design the feature by hand, but it is feature engineering: **stacking is feature engineering where the engineered feature is another model's prediction.** The hope is that a neural network, say, is not very good at the problem overall but is good precisely where the trees are weak, and giving the trees the network's guess as an input lets them use it.

Two contrasts with what came before. Unlike boosting, stacking does not change the target; y stays y throughout. Unlike bagging, the models are not independent; each sees the predictions of the ones before. And unlike both, stacking is typically used with very different model families, since that is where the new column carries information the model could not have computed itself.

What does stacking fix? Underfitting. Each new column gives the next model more to work with, so training error goes down (not necessarily strictly), and the train–test gap goes up. You stack hoping the first effect outruns the second, which is exactly the bet you make when you boost. A student asked where the overfitting comes from. Think of regression: start with one feature and two

parameters; stack ten times and the model has twelve. More parameters, same data. To see it in the extreme, imagine the stacked "predictions" are columns of pure noise, unrelated to X. A flexible model given enough noise columns will fit the training data *using the noise*, simply because it can. Real stacked predictions are far from noise, but the mechanism is the same: more possibilities to fit, no more data to fit them with.

The complete table, extending last time's:

		Training error	**Test error**	**Gap**
Bagging	increase depth	down	cannot say	up
	more trees	up	cannot say	down
Boosting	increase depth	down	cannot say	up
	more trees	down	cannot say	up
Stacking	stack more columns	down	cannot say	up

Rule of thumb: bag to reduce overfitting, generally with deep trees; boost and stack to reduce underfitting, generally boosting shallow trees, but stacking anything.

12.6 Looking ahead

The posted notebooks on bagging and boosting with XGBoost are where this becomes concrete; we start there Tuesday, and by Thursday we should be close to neural networks. The Kaggle competition runs October 11 through 16.

Lecture 13

Stacking in code, the Kaggle setup, and AUC

Tuesday, October 6, 2026

Questions of the Day

Q1. True or False: Under squared loss, the gradient of the training loss with respect to the predictions is proportional to the negative residual, so gradient boosting with squared loss recovers residual fitting.

Q2. Free answer: Do you agree that in gradient boosting, making the learning rate (step size) as small as possible always improves test performance, because smaller steps are more cautious?

Q3. True or False: After K-fold cross-validation selects the best hyperparameters, the final model should be trained on only $K-1$ folds of the data, since that is the training set size cross-validation actually evaluated.

Q4. True or False: Choosing a larger K in K-fold cross-validation makes each fold's training set more closely resemble the full data set on which the final model will be retrained, at the cost of more computation.

Q5. Free answer: Stacking appends Model 1's predictions to X as a new column before fitting Model 2. Explain in what sense this is feature engineering, and how you would implement it.

Q6. Free answer: Does stacking primarily address overfitting or underfitting?

13.1 Discussion of the questions

Kaggle is coming, and it is in effect the wrap-up of the classical half of the course: bagging, boosting, stacking, and feature engineering are what you will be using. Today we finish the review of stacking, then talk about how Kaggle works and what metric it uses.

Q1. True, and this is a discussion question wearing a true/false costume. Put the two procedures side by side. Residual fitting: `model1.fit(X, y)`; $r = y - \hat{y}$; `model2.fit(X, r)`; output $\text{model}_1 + \text{model}_2$. Gradient boosting: `model1.fit(X, y)`; compute the gradient G of the loss with respect to $\hat{y}$; `model2.fit(X, G)`; output $\text{model}_1 - \eta\,\text{model}_2$. The first steps agree. The outputs differ by the step size η and a sign. So the question is whether G and r agree.

To compute G you have to write the loss out and know what you are differentiating with respect to. For one data point, the loss is $(y-\hat{y})^2$, and the variable we can change is $\hat{y}$, not y. The derivative with respect to $\hat{y}$ is $-2(y-\hat{y})$ by the chain rule: $G = -2r$. So Model 2 is asked to fit -2 times the residual, and then it is subtracted with weight η, giving $\text{model}_1 + 2\eta\,\text{model}_2'$ where model_2' fits the

residual. With $\eta = \frac{1}{2}$ the two procedures are identical; with any other η, gradient boosting is still fitting the residual, just adding it with a different step size. (In our earlier presentation of residual fitting and exponential-loss boosting we were implicitly using a step size of 1.) The point of gradient boosting is that the residual trick only exists for additive losses, while the gradient exists for any differentiable loss.

Q2. No, and this is why it is a discussion question rather than true/false: the statement does not say what else is held fixed. Shrink η without adding trees and you simply stop making progress. Shrink η and add proportionally more trees and you pay in computation, and you also raise the risk of overfitting, because more trees is more trees. So step size and number of estimators have to be chosen together, and the way to choose them is cross-validation.

Q3. False. Once you have chosen hyperparameters, retrain on all the data. Any model, good or bad, tends to do better when trained on more data. (Why? If you throw away half your training points, training accuracy goes *up*: fewer things to fit. In the limit of one point you fit it perfectly. Test accuracy goes down.) But notice what this creates: a mismatch between how we selected and what we submit. Cross-validation with K folds judged the hyperparameters on models trained with a fraction $(K-1)/K$ of the data; the final model sees all of it.

Q4. True, and the mismatch in Q3 has a predictable direction, which is worth understanding even if you never act on it. Suppose you are choosing between 50, 75, and 100 boosted trees with 5-fold cross-validation. Each candidate is evaluated when trained on 80% of the data, and a model trained on less data overfits more. Cross-validation sees that extra overfitting and compensates: it leans toward fewer trees, shallower trees, and *more* regularization than would be best for the final model trained on 100%. Think of the extreme: with a huge amount of data, a degree-10 polynomial fit to a truly linear relationship has nothing to gain by wiggling, so it gives up and behaves linearly; with very little data it wiggles through every point. So small K *underestimates* the best number of estimators and depth, and *overestimates* the best regularization constants (λ for L1, μ for L2, or whatever letters the package uses). The effect shrinks as K grows, at the cost of computation. I would still do what cross-validation says rather than second-guess it, but it is good to know which way it is biased, and on Kaggle you do get a limited form of validation on the full-data model, which we come to below.

Q5. The first feature engineering we did was adding a column $x_1^2 + x_2^2$: a function of the existing columns, computed and appended. Stacking appends a column that is also a function of the existing columns, namely another model's prediction; the function was learned rather than written down, but it is the same move. The implementation question is where the class had to be careful, so let us write it out in the Kaggle setting, where you are given X_{train}, y_{train}, X_{test} and must produce $\hat{y}_{\text{test}}$. Take Model 1 to be a depth-6 tree and Model 2 to be logistic regression.

1. `model1.fit(X_train, y_train)`.
2. `yhat_train = model1.predict_proba(X_train)` (or `predict`; either is a legitimate column). This has one entry per training row, so it stacks: `X_train_new = [X_train | yhat_train]`.
3. `model2.fit(X_train_new, y_train)`. The target is y, not $\hat{y}$. We are not asking Model 2 to imitate Model 1; we are asking it to predict the truth with one more feature available.
4. Now the step that was missed on the first try. You cannot call `model2.predict(X_test)`: Model 2 was trained on 51 columns and X_{test} has 50, so Python will refuse (one of the rare times it tells you something is wrong rather than quietly returning nonsense). You must build the same extra column for the test data, using the already-trained Model 1: `yhat_test = model1.predict_proba(X_test)`, then `X_test_new = [X_test | yhat_test]`.

5. `yhat_test_final = model2.predict_proba(X_test_new)`. That is what you submit.

No retraining happens on the test data; you do not have its labels. Model 1 is trained once, on training data, and then applied to both.

A reminder, since it matters here: `predict` returns hard labels; `predict_proba` returns probabilities, one number for binary classification (the other is 1 minus it), one per class otherwise. For regression there is no difference. If you stack a three-class model you have to decide how to turn its three probabilities into columns.

Q6. Underfitting. Each stacked column gives the next model more to work with. And as with boosting, the risk is overfitting if you stack too much: keep adding columns, even noise columns, and training error eventually reaches zero.

That completes ensembling. Now, Kaggle.

13.2 How the Kaggle competition works

The data are split, and you only see part of the split. There is a training set $(X_{\text{train}}, y_{\text{train}})$, which you get. There is a test set X_{test}, which you get *without labels*. Hidden from you, the test set is itself cut into two pieces, $X_{\text{test},1}$ and $X_{\text{test},2}$; you cannot tell which rows belong to which.

You submit a prediction for every test row. Kaggle scores *only* the first piece and reports that score: this is the **public leaderboard**, which you can see all week. You may submit five times per day, and the allowance resets daily rather than accumulating. At the end of the competition your submission is scored on the second piece, which you never received any feedback on: this is the **private leaderboard**, and it is your final score.

Why the arrangement? The public leaderboard is extra information you are allowed to use, rationed so that you cannot overfit to it too badly: five looks a day is enough to tell whether an idea helped and not enough to tune to the noise. The private leaderboard is your true performance, because nothing you did could adapt to it. Usually the two are close. If you have overfit the public leaderboard, the private score can drop noticeably. This is the reason to spread your submissions across the week rather than saving them for the last night.

The problem will be binary classification on tabular data, and the classes are *imbalanced*. That raises the question of how to score it.

13.3 Scoring imbalanced classification

Cross-entropy is what we train with, but "my cross-entropy is 1.785" means nothing to an audience. (Reporting mean squared error or cross-entropy to a non-specialist audience is the most basic presentation mistake there is; say things people can connect to.) Accuracy people understand, but we saw in Lecture 4 why it fails on imbalanced data: with 99% positives, the model that says "positive" to everyone without looking is 99% accurate and useless.

What has gone wrong in that example is that it has a 100% *false positive* rate. To describe a classifier on imbalanced data we need to look at positives and negatives separately. Four rates, each with the denominator being the *true* class:

- **True positive rate** (TPR): of the points that are actually positive, the fraction the model calls positive.
- **False negative rate**: of the actual positives, the fraction the model calls negative. This is $1 - \text{TPR}$.
- **False positive rate** (FPR): of the points that are actually negative, the fraction the model calls positive.
- **True negative rate**: of the actual negatives, the fraction the model calls negative. This is $1 - \text{FPR}$.

For the say-positive-to-everyone model on 99% positives: TPR = 100% (good), false negative rate = 0 (good), FPR = 100% (bad), true negative rate = 0 (bad). Four numbers tell the story that one number hid. Note the pairing: TPR and false negative rate share a denominator and sum to one; FPR and true negative rate likewise. TPR and FPR do *not* sum to one; in this example both are 100%.

Four numbers are informative but awkward. To compare two algorithms we want one number, and **AUC** is the standard way to package these into one. The name is unhelpful: area under the curve, where the curve is the receiver operating characteristic (ROC). Here is what it actually is.

Computing AUC

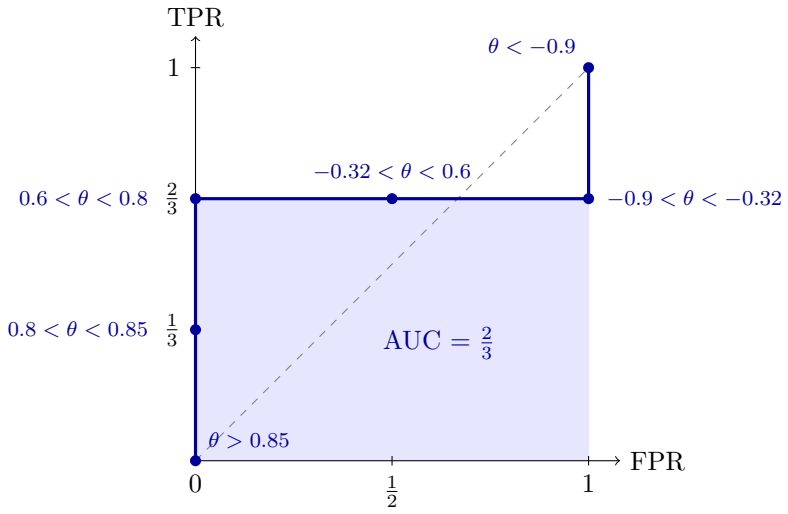
Start with the model's *scores* on a set of labeled points, not its hard labels. The convention is that a larger score means the model thinks "more likely positive." Nothing else about the scores matters: AUC does not ask whether a score of 0.3 means 50% or 90% confidence, only that 0.3 ranks above -0.1. That is what makes it applicable to any model that produces a ranking, whether trained with cross-entropy, exponential loss, or anything else.

Sort the points by score. Example, five points with scores $-0.9, -0.32, 0.6, 0.8, 0.85$ and true labels $1, 0, 0, 1, 1$ (three positives, two negatives).

Now sweep a threshold θ from left to right, calling everything above θ positive, and at each threshold record (FPR, TPR):

threshold position	FPR	TPR
below all scores (everything called positive)	2/2	3/3
between -0.9 and -0.32	2/2	2/3
between -0.32 and 0.6	1/2	2/3
between 0.6 and 0.8	0/2	2/3
between 0.8 and 0.85	0/2	1/3
above all scores (everything called negative)	0/2	0/3

The first row is always $(1, 1)$ and the last always $(0, 0)$. Plot these points with FPR on the horizontal axis and TPR on the vertical, and connect them; that is the ROC curve.



The six dots are the six rows of the table, each labeled by where the threshold sits; the shaded region is the area under the curve, and that area is the AUC. Here it is 2/3: the curve climbs to TPR $= 2/3$ while FPR is still 0, then stalls, and only reaches TPR $= 1$ after the false positive rate has gone all the way to 1. The dashed diagonal is what a model that assigns random scores would trace, AUC $= 1/2$. You will not compute these by hand in practice, but people talk about AUC constantly and you should know what it is.

What is the best possible AUC? If the model's ordering puts every positive above every negative, the curve goes straight up the left edge to $(0, 1)$ and across the top to $(1, 1)$, enclosing the whole unit square: AUC $= 1$. Scores like $0.1, 0.4$ for the negatives and $0.8, 0.9$ for the positives do this. Read what that says: the model has correctly *sorted* the patients. Every patient with the disease scored higher than every patient without it. The model has not told you where to put the threshold, and AUC will not tell you either, but it has demonstrated that the signal is there and that it found it.

The probability interpretation

There is a second way to say what AUC measures. Pick a positive point uniformly at random and a negative point uniformly at random. The AUC is the probability that the positive's score is higher than the negative's. Check it on the example: with probability 1/3 you pick the positive with score -0.9, which loses to both negatives; with probability 2/3 you pick one of the other two positives, which beat both negatives. Total: 2/3, as computed. For a model with AUC 1, a randomly chosen sick patient always outscores a randomly chosen healthy one.

13.4 Looking ahead

More on AUC and on Kaggle strategy on Thursday, and then neural networks.

Lecture 14

Kaggle strategy, more on AUC, and the first neural network

Thursday, October 8, 2026

Questions of the Day

Q1. True or False: When K-fold cross-validation uses a small K, it can select stronger regularization than would be best when the final model is retrained on the full training data.

Q2. True or False: Applying any strictly increasing transformation to a model's scores leaves its ROC curve and AUC unchanged.

Q3. True or False: The false positive rate is equal to 1 minus the true positive rate because every prediction must be either correct or incorrect.

Q4. True or False: A model can have an AUC of 1 even if its reported scores are poorly calibrated, provided every positive observation receives a higher score than every negative observation.

Q5. Free answer: In the course Kaggle setup, why are submissions scored on only part of the hidden test set during the competition, and why might the public leaderboard score differ from the final private score?

Q6. Free answer: What is the probability interpretation of AUC? What important decision about deployment does AUC not make for you?

14.1 Kaggle: a working strategy

Before the questions, the practical picture of what the week will look like. You get X_{train}, y_{train}, and X_{test}, as CSV files, and you submit a CSV of (row id, prediction). Two logistics points first.

Submit soft scores. The metric is AUC, which sweeps over all thresholds. If you submit hard labels you have already picked a threshold and thrown away the information AUC is designed to use. Concretely: `model.predict_proba`, not `model.predict`; the numbers you submit should be between 0 and 1, not equal to 0 or 1. That is the only thing about AUC you need to get right for Kaggle.

The loop. At any moment you have a current best model. The question you keep asking about it is the one we have asked all semester: am I overfitting or underfitting? The diagnosis determines what to try next.

If overfitting (large train–test gap): add or increase regularization (L1, L2, or something more creative like adding noise to the data); bag, either the same model many times or several different

models, or just average your two best; or move to a simpler family. If you are boosting depth-10 trees, do not reach for regularization; make the trees shallower.

If underfitting (training accuracy lower than you believe the data supports): remove or decrease regularization; move to a richer family; or do feature engineering, which on Kaggle is the interesting one, because you will have 20 to 30 features with no meaning attached and cannot reason your way to "$x_1^2 + x_2^2$." Stacking is one systematic way to engineer features. Another is brute force: add all pairwise products $x_i x_j$, or all triples. Count before you do this. With $d = 20$ features there are $\binom{20}{2} \approx 200$ pairs and $\binom{20}{3} \approx 1{,}100$ triples ($\binom{n}{k}$ scales roughly like $n^k/k!$). Going from 20 features to 1,100 on 10,000 to 20,000 rows will almost certainly overfit. So you want only some of the new features, and now you have a **feature selection** problem.

Feature selection

Suppose you have 1,000 candidate features and believe only a handful matter. Trying every subset of four is $\binom{1000}{4}$, on the order of $10^{12}/24$; not happening. I like brute force when it is affordable and avoid it when it isn't, so the question is how to cut the candidate list down to something you can brute-force, or sidestep the search entirely. Ideas from class, none of them guaranteed to work, all of them worth trying:

L1 regularization. We saw in the lasso notebook that an L1 penalty produces sparse solutions, using only a few features and setting the rest to exactly zero. That is a feature selector you already know.

Correlation. Compute each feature's correlation with y and drop the ones that look like noise. This was a homework question (which features of `patient.csv` matter?), and it generalizes: identify noisy or unrelated columns and remove them. It is a dangerous game, since you might discard real information, but cutting from 1,000 columns to 100 is a large reduction in your capacity to overfit, and Kaggle will tell you whether it helped.

Greedy forward selection. You cannot find the best four features out of 1,000, but you can find the best *one*: build a one-column X, run `fit` and evaluate on a held-out set, and repeat for each of the 1,000 columns. A thousand fits of a small model takes a minute or two. Say column 17 wins. Freeze it and search for the column that best complements it: another 999 fits. Four rounds gives you four features for about 4,000 fits instead of 10^{12}. The best four features might be ones that are individually terrible and only useful together, and greedy search will miss that. We accept it, because the alternative is not computable.

Data preprocessing

Before fitting anything, look at the data. With 20 to 30 features you can look at every pairwise correlation and every histogram, which you could not do with 10,000 features. Things to look for:

Categorical features in disguise. A column might contain only the values 78704, 78751, 78703, 78752. Those are numbers, but they are zip codes; that 78751 is larger than 78704 means nothing, while "this one is Tarrytown and that one is central Austin" means a lot. You will only notice if you plot the column and see four spikes. The standard fix is **one-hot encoding**: replace the column by four 0/1 columns, one per value, with a single 1 in each row. (This throws away any possibility of learning that two zip codes are related; we will revisit that when we get to embeddings, where the question is how to do this better when there are thousands of categories.)

Outliers and skew. Are there a few rows far outside the range of the rest? Is a feature bimodal? Decide what to do about it, as we discussed on day one with the zero-BMI patients.

Then start building. Get on the board immediately with a vanilla XGBoost and a vanilla random forest from `sklearn`, no hyperparameter tuning. Logistic regression alone will not do well. From there, iterate: diagnose, pick a strategy from the list, test, repeat. Static resources such as Kaggle forums are fine to consult. The competition is individual, runs a week, and afterwards you will submit a short description of what you did plus the code that generates your best submission (things that did not work are welcome too). There is no theory telling you what will work on a given dataset; the point is to find out by trying.

14.2 Discussion of the questions

Q1. True. If you had ten times the data, would you need more regularization or less? Less; with infinite data you cannot overfit and need none. Small K means each fold trains on a smaller fraction of the data, say two-thirds for $K = 3$, so cross-validation is answering "what regularization is best when training on two-thirds of the data," and that answer is more regularization than is best for the full set. The same reasoning makes it favor slightly simpler models generally. The effect is modest, it shrinks as K grows, and I am not suggesting you override cross-validation on this account; just know which way it leans.

Q2. True. Take scores $-0.6, 0.1, 0.7, 0.9$ and transform them to $-0.1, 0.4, 0.72, 0.99$. A strictly increasing transformation cannot swap any two points. The ROC curve is built by sweeping a threshold through the sorted scores and recording which points are called positive at each step; if the order is unchanged, the sequence of hard labelings is unchanged, so every (FPR, TPR) pair is unchanged, so the curve and its area are unchanged. AUC depends only on the *ranking*.

Q3. False. FPR is (negatives called positive)/(total negatives); TPR is (positives called positive)/(total positives). Different denominators, and no reason to sum to one. The model that calls everything positive has TPR $= 1$ and FPR $= 1$, which is the first point of every ROC curve. What *is* true is TPR + FNR $= 1$ and FPR + TNR $= 1$.

Q4. True, and the example is worth seeing because it shows how different AUC is from accuracy. Eight points, listed in order of score:

score	0.10	0.11	0.11	0.15	0.20	0.22	0.30	0.40
true label	0	0	0	0	1	1	1	1

Round at 0.5 and every prediction is 0: accuracy is $4/8 = 50\%$, random guessing. Now trace the ROC curve. Threshold below everything: $(1, 1)$. Step past 0.10: FPR 3/4, TPR 1. Past 0.11 (both): FPR 1/2 then 1/4, TPR 1. Past 0.15: FPR 0, TPR 1. The curve has gone straight across the top to $(0, 1)$ and will now go straight down the left edge. AUC $= 1$.

Is that a defect of AUC? I would say the opposite. The model has sorted the patients perfectly: every patient with the disease scored higher than every patient without it. It found the signal. The only thing wrong is a misplaced threshold, and 0.5 was never a law of nature. A blood test does not tell you whether you have the disease; it gives you a number, and the question of where to draw the line for further testing is a separate decision. A test that perfectly ranks sick above healthy is an excellent test, and AUC $= 1$ says so. Accuracy at 0.5 says it is coin-flipping. AUC is telling the truth here.

Q5. Scoring only part of the hidden test set during the week guarantees that you can never overfit to the other part, because you never get any feedback on it. The public score is still possible to overfit, slowly, even at five submissions a day with nothing visible but a number, so your private score can differ from your public score; the private score is the more honest estimate precisely because you could not adapt to it.

Q6. Pick a random positive and a random negative; AUC is the probability that the positive's score exceeds the negative's. AUC $= 1$ means every positive outscores every negative: two people take the same test, and the one with the disease always gets the higher number. What AUC does *not* do is choose the threshold. It tells you the model can separate the classes, not where to cut when you deploy it, and that decision depends on the relative costs of false positives and false negatives, which are not in the data.

14.3 Neural networks: the first building block

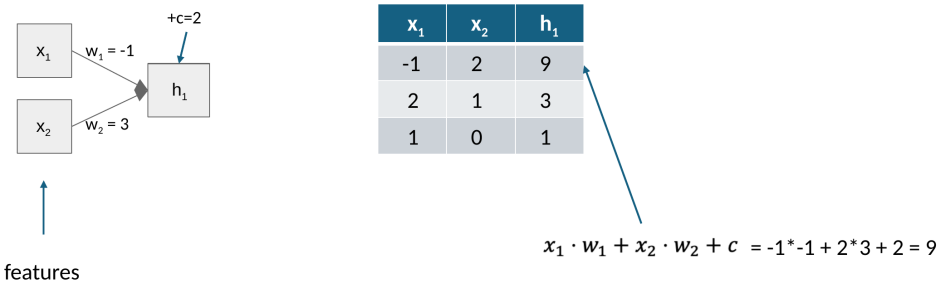
Everyone here has trained a neural network before, in one class or another. We are going to rebuild them from the ground up, as modular blocks, because that is how they are designed and how they are written in code. Today, the first block. Next time, why it is not enough.

A fully connected layer

The basic unit takes inputs and combines them by multiplying by weights and adding a constant. With two features,

$$h_1 = w_1 x_1 + w_2 x_2 + c.$$

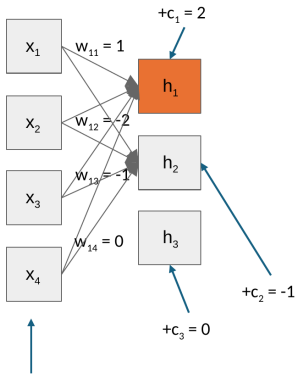
This is linear regression, drawn differently. With $w_1 = -1$, $w_2 = 3$, $c = 2$ and inputs $(x_1, x_2) = (-1, 2)$: $h_1 = 1 + 6 + 2 = 9$. Nothing new.



x_1	x_2	h_1
-1	2	9
2	1	3
1	0	1

One unit: two inputs, two weights, a constant. The table evaluates it on three input rows. (From the lecture slides.)

The point is that it is a block, and blocks stack. Four inputs can feed three such units, each with its own weights w_{jk} and constant c_j. With weights $(1, -2, -1, 0)$ and constant 2 on inputs $(0, 2, 1, 4)$: $h_1 = 0 - 4 - 1 + 0 + 2 = -3$, and so on for h_2, h_3 with their own weights.



x_1	x_2	x_3	x_4	h_1
0	2	1	4	-3
-1	-1	2	-3	1
0	3	0	2	-4

A fully connected layer: four inputs to three units. Only the arrows into h_1 are drawn; each of h_2 and h_3 has its own four weights and constant. The table shows h_1 on three input rows.

The outputs h_1, h_2, h_3 are just numbers, so nothing stops us from feeding them into another layer of three units, and those into a final unit that produces y.

Vocabulary: when every input connects to every unit, the layer is **fully connected** (PyTorch calls it `Linear`, a worse name, since nearly everything we build will be linear layers of one sort or another). The intermediate values are **hidden layers**, and the individual units are sometimes called **neurons**; hidden because we see the input and the output and these are internal to the computation. The number of units in a layer is its **width**; the number of layers is its **depth**. The input having four features does not constrain the first hidden layer to have four units; it can be wider or narrower.

Counting parameters

Take the network $4 \to 3 \to 3 \to 1$ with all connections present. First layer: 4 inputs to each of 3 units is 12 weights, plus 3 constants, 15. Second layer: $3 \times 3 = 9$ weights plus 3 constants, 12. Output: 3 weights, no constant as drawn, 3. Total 30 parameters.

Compare linear regression on the same four features: $y = w_0 + w_1x_1 + \cdots + w_4x_4$, five parameters. Linear regression is the map from the four inputs straight to one output unit. The network is three different regressions on the inputs, then three regressions on *those*, then one regression on those. It describes a potentially much more complicated function of $x_1, \ldots, x_4$, with 30 parameters instead of 5. If you cranked through the algebra you could write y directly as a function of the x's and the w's and c's, with no h's appearing; the h's are scaffolding.

Why add layers? Same answer as always: when you are underfitting. More width and more depth means more parameters, hence more flexibility, hence (potentially) less underfitting and more overfitting.

Classification: softmax

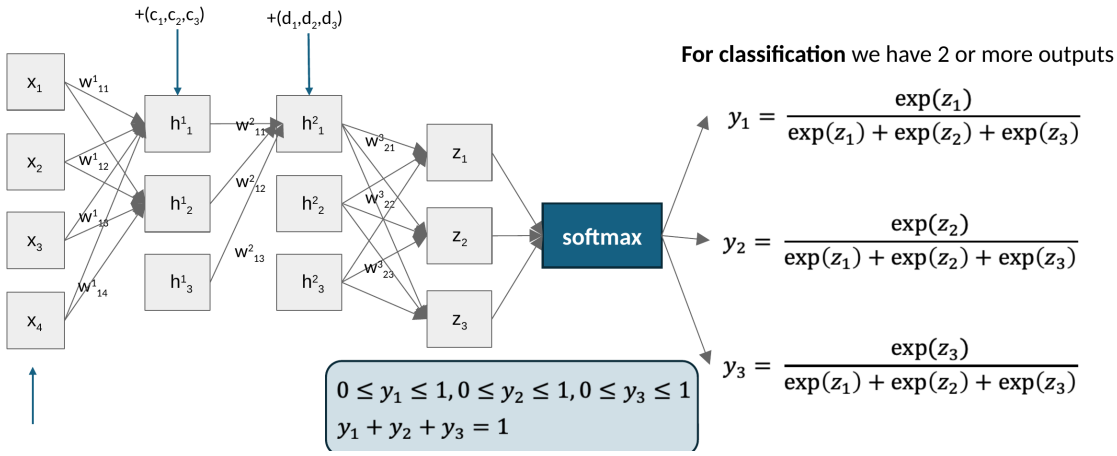
For regression the network ends in one output. For classification into k classes, MNIST or CIFAR-10, it ends in k outputs, and we want them to be probabilities. We have met this problem: multiplying and adding will never guarantee numbers between 0 and 1 that sum to 1. For two classes the sigmoid fixed it. For k classes the fix is **softmax**. Let the k outputs of the last linear layer be scores

$z_1, \ldots, z_k$, called **logits** as before, and set

$$y_j = \frac{e^{z_j}}{e^{z_1} + \cdots + e^{z_k}}.$$

Each e^{z_j} is strictly positive, so each $y_j > 0$. The numerator appears in the denominator alongside other positive terms, so each $y_j < 1$. And the y_j sum to one by construction. Softmax also preserves order: if $z_1 < z_3 < z_2$ then $y_1 < y_3 < y_2$. A larger score means a larger probability, which is the only meaning the scores had in the first place: z_1 growing means the model is more confident the answer is class 1, exactly as with exponential loss.

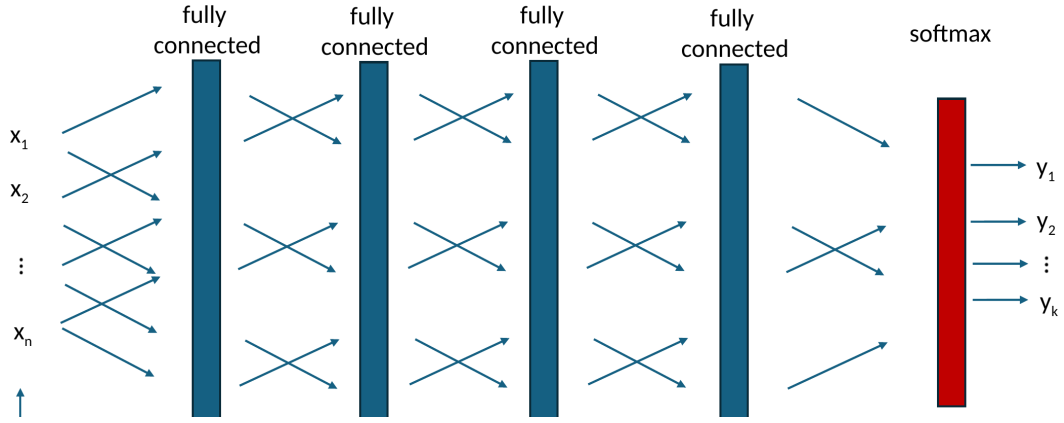
Softmax has no parameters. Like the sigmoid, it is for us. If the network outputs scores $(3, 3.3, -2)$ for dog, cat, horse, I cannot interpret that; "42% dog, 57% cat, 0% horse" I can. When we get to training we will often apply the loss directly to the logits.



The full picture for three-class classification: $4 \to 3 \to 3 \to 3$ fully connected layers produce logits z_1, z_2, z_3, and softmax turns them into probabilities.

The compact notation

Rather than drawing boxes and arrows, a network of this kind is fully specified by a list: n input features, then a fully connected layer with m_1 units, then one with m_2, ..., then k outputs, then softmax if classifying. A few symbols describe a model with thousands or millions of parameters. That compactness is the point of thinking in modules.

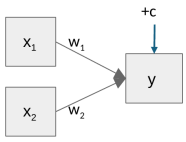


The same idea as a block diagram: features in, a chain of fully connected layers, softmax out.

14.4 A question to end on

We can now build networks of any width and depth, from any number of inputs, for regression or for classification. Is that enough? Here is the test. The XOR function on two binary inputs outputs 1 when exactly one input is 1, and 0 when both are 0 or both are 1: four points, $(0,0) \to 0$, $(1,1) \to 0$, $(0,1) \to 1$, $(1,0) \to 1$. Can a network built from the blocks above learn it, with zero training error?

X1	X2	Y
0	0	0
0	1	1
1	0	1
1	1	0



The XOR data and the one-layer network $y = w_1x_1 + w_2x_2 + c$ asked to fit it.

The answer is going to be no, however wide or deep, and understanding why reveals the one ingredient we are missing. Tuesday.